FIAP ON

TRATORES DIGITAIS - HARDWARES PARA IA

TORNANDO AS CONSULTAS MAIS INTELIGENTES E EFICAZES

4

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Resultado da consulta à tabela FUNCIONARIO 20
Figura 2 - Resultado da consulta à tabela FUNCIONARIO 21
Figura 3 - Resultado da consulta à tabela FUNCIONARIO (Select/Where) 22
Figura 4 - Resultado da consulta à tabela FUNCIONARIO (Select/Distinct) 23
Figura 5 - Resultado da consulta à tabela FUNCIONARIO, comando SELECT com operador aritmético 24
Figura 6 - Resultado da consulta à tabela FUNCIONARIO com ALIAS de colunas..25
Figura 7 - Resultado da consulta à tabela DEPARTAMENTO – ORDER BY.....26
Figura 8 - Resultado da consulta à tabela DEPARTAMENTO – ORDER BY.....27
Figura 9 - Resultado da consulta à tabela FUNCIONARIO – ORDER BY 28
Figura 10 - Resultado da consulta à tabela FUNCIONARIO – ORDER BY 29
Figura 11 - Operadores Relacionais.....30
Figura 12 - Resultado da consulta à tabela FUNCIONARIO – Operador Relacional 31
Figura 13 - Resultado da consulta à tabela FUNCIONARIO – Operador Relacional 31
Figura 14 - Resultado da consulta à tabela FUNCIONARIO – Operador Relacional 32
Figura 15 - Resultado da consulta à tabela FUNCIONARIO – Operador Relacional 33
Figura 16 - Operadores LÓGICOS.....34
Figura 17 - Resultado da consulta à tabela FUNCIONARIO – Operador Lógico 34
Figura 18 - Resultado da consulta à tabela FUNCIONARIO – Operador Lógico 35
Figura 19 - Resultado da consulta à tabela FUNCIONARIO – Operador Lógico 36
Figura 20 - Operadores Especiais ou Operadores SQL 37
Figura 21 - Resultado da consulta à tabela FUNCIONARIO – Operador Between...38
Figura 22 - Resultado da consulta à tabela FUNCIONARIO – Operador Between...38
Figura 23 - Resultado da consulta à tabela FUNCIONARIO – Operador IN() 39
Figura 24 - Resultado da consulta à tabela FUNCIONARIO – Operador NOT IN() ..40
Figura 25 - Resultado da consulta à tabela FUNCIONARIO – Operador IN() 40
Figura 26 - Resultado da consulta à tabela FUNCIONARIO – Operador IS NOT NULL 41
Figura 27 - Resultado da consulta à tabela FUNCIONARIO – Operador IS NULL ...42
Figura 28 - Operadores Especiais ou Operadores SQL 43
Figura 29 - Resultado da consulta à tabela DEPARTAMENTO – Operador LIKE 43
Figura 30 - Resultado da consulta à tabela DEPARTAMENTO – Operador LIKE 44
Figura 31 - Resultado da consulta à tabela DEPARTAMENTO – Operador LIKE 44
Figura 32 - Resultado da consulta à tabela DEPARTAMENTO – Operador LIKE 45
Figura 33 - Resultado da consulta à tabela FUNCIONARIO – Vários operadores....46
Figura 34 - Resultado da consulta à tabela FUNCIONARIO – operador de concatenação 47
Figura 35 - Resultado da consulta à tabela DEPARTAMENTO – pseudocoluna ROWNUM 48
Figura 36 - Resultado da consulta à tabela FUNCIONARIO – pseudocoluna ROWNUM 48
Figura 37 - Resultado da consulta à tabela FUNCIONARIO – análise TOP-N.....50
Figura 38 - Resultado da consulta à tabela V$NLS_PARAMETERS – atributos NLS 50
Figura 39 - Resultado da consulta à tabela PROJETO após alteração do NLS_DATE_FORMAT.....51
Figura 40 - Constantes para configuração do atributo NLS_DATE_FORMAT 52

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 41 - Resultado da consulta a data do sistema após alteração do NLS_DATE_LANGUAGE 53
Figura 42 - Resultado da Consulta 1 55
Figura 43 - Resultado da Consulta 2 56
Figura 44 - Resultado da Consulta 3 57
Figura 45 - Resultado da Consulta 4 58
Figura 46 - Resultado da Consulta 5 59
Figura 47 - Resultado da Consulta 6 60

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Quadro 1 - Operadores Aritméticos.....23

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Exemplo de código-fonte SQL – comando SELECT 9
Código-fonte 2 - Script de criação do banco de dados.....12
Código-fonte 3 - Script para popular as tabelas do banco de dados SIP 19
Código-fonte 4 - Exemplo de código-fonte SQL – comando SELECT 19
Código-fonte 5 - Exemplo de código-fonte SQL – comando SELECT 20
Código-fonte 6 - Exemplo de código-fonte SQL – comando SELECT com WHERE 21
Código-fonte 7 - Exemplo de código-fonte SQL – comando SELECT, com DISTINCT 22
Código-fonte 8 - Exemplo de código-fonte SQL – comando SELECT com operador aritmético.....24
Código-fonte 9 - Exemplo de código-fonte SQL – comando SELECT com ALIAS de colunas.....25
Código-fonte 10 - Exemplo de código-fonte SQL – comando SELECT com ORDER BY 26
Código-fonte 11 - Exemplo de código-fonte SQL – comando SELECT com ORDER BY 27
Código-fonte 12 - Exemplo de código-fonte SQL – comando SELECT com ORDER BY 28
Código-fonte 13 - Exemplo de código-fonte SQL – comando SELECT com ORDER BY 29
Código-fonte 14 - Exemplo de código-fonte SQL – comando SELECT com operadores relacionais 30
Código-fonte 15 - Exemplo de código-fonte SQL – comando SELECT com operadores relacionais 31
Código-fonte 16 - Exemplo de código-fonte SQL – comando SELECT com operadores relacionais 32
Código-fonte 17 - Exemplo de código-fonte SQL – comando SELECT com operadores relacionais 33
Código-fonte 18 - Exemplo de código-fonte SQL – comando SELECT com operadores lógicos 34
Código-fonte 19 - Exemplo de código-fonte SQL – comando SELECT com operadores lógicos 35
Código-fonte 20 - Exemplo de código-fonte SQL – comando SELECT com operadores lógicos 36
Código-fonte 21 - Exemplo de código-fonte SQL – comando SELECT com operador BETWEEN.....37
Código-fonte 22 - Exemplo de código-fonte SQL – comando SELECT com operador BETWEEN.....38
Código-fonte 23 - Exemplo de código-fonte SQL – Comando SELECT com operador IN() 39
Código-fonte 24 - Exemplo de código-fonte SQL – Comando SELECT com operador NOT IN().....40
Código-fonte 25 - Exemplo de código-fonte SQL – Comando SELECT com operador IN() 40
Código-fonte 26 - Exemplo de código-fonte SQL – comando SELECT com operador IS NOT NULL 41

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Código-fonte 27 - Exemplo de código-fonte SQL – comando SELECT com operador IS NULL....42
Código-fonte 28 - Exemplo de código-fonte SQL – comando SELECT com operador LIKE....43
Código-fonte 29 - Exemplo de código-fonte SQL – comando SELECT com operador LIKE....44
Código-fonte 30 - Exemplo de código-fonte SQL – comando SELECT com operador LIKE....44
Código-fonte 31 - Exemplo de código-fonte SQL – comando SELECT com operador LIKE....45
Código-fonte 32 - Exemplo de código-fonte SQL – comando SELECT, vários operadores....46
Código-fonte 33 - Exemplo de código-fonte SQL – comando SELECT com operador de concatenação....47
Código-fonte 34 - Exemplo de código-fonte SQL – comando SELECT, pseudocoluna ROWNUM....48
Código-fonte 35 - Exemplo de código-fonte SQL – comando SELECT, pseudocoluna ROWNUM....48
Código-fonte 36 - Exemplo de código-fonte SQL – comando SELECT, análise TOP-N....49
Código-fonte 37 - Exemplo de código-fonte SQL – comando SELECT, atributos NLS....50
Código-fonte 38 - Exemplo de código-fonte SQL – comando SELECT, atributos NLS_DATE_FORMAT....51
Código-fonte 39 - Exemplo de código-fonte SQL – comando SELECT....51
Código-fonte 40 - Exemplo de código-fonte SQL – comando SELECT, atributos NLS_DATE_LANGUAGE....52
Código-fonte 41 - Exemplo de código-fonte SQL – comando SELECT....52
Código-fonte 42 - Exemplo de código-fonte SQL – comando SELECT – Consulta 1....55
Código-fonte 43 - Exemplo de código-fonte SQL – comando SELECT – Consulta 2....55
Código-fonte 44 - Exemplo de código-fonte SQL – comando SELECT – Consulta 3....56
Código-fonte 45 - Exemplo de código-fonte SQL – comando SELECT – Consulta 4....57
Código-fonte 46 - Exemplo de código-fonte SQL – comando SELECT – Consulta 5....58
Código-fonte 47 - Exemplo de código-fonte SQL – comando SELECT – Consulta 6....60

FIXAD

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 TORNANDO AS CONSULTAS MAIS INTELIGENTES E EFICAZES 8
1.1 Divisão DRS ou DQL 8
1.1.1 Comando SELECT 9
1.2 Exemplos de aplicação do comando SELECT 10
1.3 Consulta Simples 19
1.3.1 Exemplo de comando SELECT em consultas simples 19
1.3.2 Exemplo de comando SELECT com declaração de colunas 20
1.3.3 Exemplo de comando SELECT com declaração de colunas e condição (filtro) 21
1.3.4 Exemplo de comando SELECT com a opção DISTINCT 22
1.4 Operadores Aritméticos 23
1.5 ALIAS – Apelido para colunas 24
1.6 Cláusula ORDER BY 26
1.7 Consulta em tabelas utilizando filtros 29
1.7.1 Operadores relacionais 30
1.7.2 Operadores lógicos 33
1.7.3 Operadores Especiais ou operadores SQL 36
1.7.4 Exemplo de consulta com vários operadores 45
1.8 Operador de concatenação (||) 46
1.9 Pseudocoluna ROWNUM 47
1.10 Análise TOP-N 49
1.11 Configurando parâmetros NLS (National Language Support) 50
2 HANDS ON – CONSULTA OU RECUPERAÇÃO DE DADOS 54
2.1 Estudo de Caso: SIP – SISTEMA DE CONTROLE DE IMPLANTACAO DE PROJETOS 54
2.1.1 Recuperando dados com comandos DRS ou DQL 54
REFERÊNCIAS 61
GLOSSÁRIO 62

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 TORNANDO AS CONSULTAS MAIS INTELIGENTES E EFICAZES

A principal função de um sistema de banco de dados é proporcionar recursos para armazenamento, manutenção e consulta dos dados, de maneira segura e eficaz.

A **consulta de dados é uma pesquisa** realizada **no banco** de dados, cujo propósito é recuperar as linhas (registros) **que atendem** às **condições pré-estabelecidas**. As consultas podem envolver diferentes recursos e, por isso, apresentam as seguintes definições:

- Consultas simples;
- Consultas baseadas em condições simples ou compostas;
- Consultas que recuperam dados de diferentes tabelas (junções);
- Consultas que utilizam funções para realização de diferentes operações;
- Consultas que manipulam grupos de dados;
- Consultas baseadas no resultado de outras consultas.

A instrução utilizada para a realização das consultas é o **SELECT**, que faz parte da categoria de operações DML (Data Manipulation Language).

Alguns autores criam um grupo à parte para tratar somente do SELECT, o **DQL**, Data Query Language (**Linguagem de Consulta de Dados**) ou **DRS**, Data Retrieve Language (**Linguagem de Recuperação de Dados**).

1.1 Divisão DRS ou DQL

Na linguagem SQL temos instruções que nos permitem recuperar os dados de tabelas de um banco de dados. Essas instruções pertencem à divisão DRS (*Data Retrieve Statement*) ou DQL (*Data Query Language*) da linguagem SQL.

São operações de recuperação de dados de tabelas de um banco de dados:

- SELECT simples.
- SELECT com operadores.
- SELECT a partir de junções.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- SELECT a partir de agrupamentos.

Após a implementação da estrutura de um banco de dados e de ter populado inicialmente a estrutura de armazenamento, o passo seguinte é recuperar os dados das tabelas de um banco de dados, a partir das necessidades dos usuários.

Recuperamos os dados das tabelas através do comando SELECT.

1.1.1 Comando SELECT

Para recuperarmos os dados das tabelas de um banco de dados, utilizamos o comando SELECT, que é um comando dentro da divisão DRS/DQL da linguagem SQL.

Sintaxe básica do comando SELECT:

```
      SELECT [ DISTINCT | ALL] { * | coluna  [, coluna, … ] }
        FROM tabela
       WHERE condição
    ORDER BY coluna [, coluna, ...]
```

Código-fonte 1 - Exemplo de código-fonte SQL – comando SELECT
Fonte: Price (2009)

Em que:

- **SELECT:** especifica as colunas (campos) necessários para a pesquisa.
- **DISTINCT:** não mostra eventuais valores repetidos de colunas.
- **ALL:** mostra todos os valores, mesmo que repetidos; esse é o padrão se o DISTINCT não for definido, portanto, não precisamos escrevê-lo.
- *** (asterisco):** indica que devem ser mostradas todas as colunas da tabela.
- **FROM:** indica em que tabelas serão efetuadas estas pesquisas.
- **WHERE:** condição para que se execute a pesquisa (filtra dados).
- **ORDER BY:** especifica em que ordem deverá ser apresentada a pesquisa desejada; por qual campo estará ordenada (de forma crescente ou decrescente).
- **<nome-tabela>:** tabela que terá os dados recuperados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **<nome-coluna>:** colunas que serão recuperadas da tabela.
- **<condição>:** condição (filtro) utilizado no momento da recuperação dos dados da tabela.

1.2 Exemplos de aplicação do comando SELECT

Para exemplificar as aplicações dos comandos DRS/DQL, iremos construir (implementar) e popular uma estrutura de banco de dados.

Como sugestão, podemos utilizar a ferramenta SQL Developer da Oracle.

O nome do SCRIPT (arquivo com comandos SQL) deve possuir um nome que ajude na sua identificação, por exemplo: “SCRIPT_DDL_PROJETO_SIP.SQL”, com a extensão SQL.

A seguir, apresentamos o código na linguagem SQL, que permite implementar a estrutura de banco de dados do PROJETO SIP.

```
-- SCRIPT PARA CRIAÇÃO DO BANCO DE DADOS

-- DO ESTUDO DE CASO SIP

-- COMANDOS PARA EXCLUIR AS TABELAS

DROP TABLE t_sip_departamento CASCADE CONSTRAINTS;
DROP TABLE t_sip_dependente CASCADE CONSTRAINTS;
DROP TABLE t_sip_funcionario CASCADE CONSTRAINTS;
DROP TABLE t_sip_implantacao CASCADE CONSTRAINTS;
DROP TABLE t_sip_projeto CASCADE CONSTRAINTS;

-- COMANDOS PARA CRIAR AS TABELAS

-- E ADICIONAR CONSTRAINTS: PRIMARY KEY, CHECK E UNIQUE

CREATE TABLE t_sip_departamento (
    cd_depto NUMBER(3)       NOT NULL,
    nm_depto VARCHAR2(30)    NOT NULL,
    sg_depto CHAR(3)         NOT NULL
);

COMMENT ON COLUMN t_sip_departamento.cd_depto IS
    'Um departamento é identificado por um código numérico de
três dígitos.';

   ALTER TABLE t_sip_departamento
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
ADD CONSTRAINT pk_sip_departamento
   PRIMARY KEY ( cd_depto );

   ALTER TABLE t_sip_departamento
ADD CONSTRAINT un_sip_depto_nome
        UNIQUE ( nm_depto );

   ALTER TABLE t_sip_departamento
ADD CONSTRAINT un_sip_depto_sigla
        UNIQUE ( sg_depto );

CREATE TABLE t_sip_dependente (
    nr_matricula  NUMBER(5)     NOT NULL,
    cd_dependente NUMBER(2)     NOT NULL,
    nm_dependente VARCHAR2(50)  NOT NULL,
    dt_nascimento DATE          NOT NULL
);

   ALTER TABLE t_sip_dependente
ADD CONSTRAINT pk_sip_dependente
   PRIMARY KEY ( nr_matricula, cd_dependente );

CREATE TABLE t_sip_funcionario (
    nr_matricula      NUMBER(5)     NOT NULL,
    cd_depto          NUMBER(3)     NOT NULL,
    nm_funcionario    VARCHAR2(50)  NOT NULL,
    dt_nascimento     DATE          NOT NULL,
    dt_admissao       DATE          NOT NULL,
    ds_endereco       VARCHAR2(100) NOT NULL,
    vl_salario_mensal NUMBER(7, 2) NOT NULL
);

   ALTER TABLE t_sip_funcionario
ADD CONSTRAINT ck_sip_func_salario
         CHECK ( vl_salario_mensal >= 1212 );

   ALTER TABLE t_sip_funcionario
ADD CONSTRAINT pk_sip_funcionario
   PRIMARY KEY ( nr_matricula );

CREATE TABLE t_sip_implantacao (
    cd_projeto     NUMBER(5) NOT NULL,
    cd_implantacao NUMBER(3) NOT NULL,
    nr_matricula   NUMBER(5) NOT NULL,
    dt_entrada     DATE      NOT NULL,
    dt_saida       DATE          NULL
);

   ALTER TABLE t_sip_implantacao
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
ADD CONSTRAINT ck_sip_implant_data
         CHECK ( dt_saida > dt_entrada );

   ALTER TABLE t_sip_implantacao
ADD CONSTRAINT pk_sip_implantacao
   PRIMARY KEY ( cd_projeto, cd_implantacao );

CREATE TABLE t_sip_projeto (
    cd_projeto NUMBER(5)    NOT NULL,
    nm_projeto VARCHAR2(40) NOT NULL,
    dt_inicio  DATE         NOT NULL,
    dt_termino DATE
);

-- ADICIONAR CONSTRAINT FOREIGN KEY

   ALTER TABLE t_sip_projeto
ADD CONSTRAINT ck_sip_projeto_data
         CHECK ( dt_termino > dt_inicio );

   ALTER TABLE t_sip_projeto
ADD CONSTRAINT pk_sip_projeto
   PRIMARY KEY ( cd_projeto );

   ALTER TABLE t_sip_projeto
ADD CONSTRAINT un_sip_projeto_nome
        UNIQUE ( nm_projeto );

   ALTER TABLE t_sip_dependente
ADD CONSTRAINT fk_sip_dependente_func
   FOREIGN KEY ( nr_matricula )
    REFERENCES t_sip_funcionario ( nr_matricula );

   ALTER TABLE t_sip_funcionario
ADD CONSTRAINT fk_sip_func_depto
   FOREIGN KEY ( cd_depto )
    REFERENCES t_sip_departamento ( cd_depto );

   ALTER TABLE t_sip_implantacao
ADD CONSTRAINT fk_sip_implant_func
   FOREIGN KEY ( nr_matricula )
    REFERENCES t_sip_funcionario ( nr_matricula );

   ALTER TABLE t_sip_implantacao
ADD CONSTRAINT fk_sip_implant_projeto
   FOREIGN KEY ( cd_projeto )
    REFERENCES t_sip_projeto ( cd_projeto );
```

Código-fonte 2 - Script de criação do banco de dados
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Agora, vamos popular toda a estrutura do banco de dados referente ao projeto SIP.

O nome do SCRIPT (arquivo com comandos SQL) deve possuir um nome que ajude na sua identificação, por exemplo: “SCRIPT_DML_PROJETO_SIP.SQL”, com a extensão SQL.

A seguir, apresentamos o código na linguagem SQL, que permite popular a estrutura de banco de dados do PROJETO SIP.

```
-- SCRIPT PARA POPULAR O BD IMPLANTACAO DE PROJETOS

-- ELIMINA O CONTEUDO DAS TABELAS
/*
DELETE FROM T_SIP_IMPLANTACAO;
DELETE FROM T_SIP_PROJETO;
DELETE FROM T_SIP_DEPENDENTE;
DELETE FROM T_SIP_FUNCIONARIO;
DELETE FROM T_SIP_DEPARTAMENTO;
*/

-- POPULAR A TABELA DEPTO COM 5 DEPARTAMENTOS
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (1, 'FINANCEIRO', 'FIN');
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (2, 'TECNOLOGIA DA INFORMAÇÃO','TIN');
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (3, 'CONTAS A PAGAR','CPG');
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (4, 'FATURAMENTO','FAT');
INSERT INTO T_SIP_DEPARTAMENTO
     VALUES (5, 'RECURSOS HUMANOS', 'RHU');

-- POPULAR A TABELA FUNCIONARIO COM 3 FUNCIONARIOS PARA CADA
DEPTO
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- DEPTO = 1
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12345,         1,         'JOAO         DA         SILVA',
TO_DATE('10/05/1985','DD/MM/YYYY'),
     TO_DATE('15/09/2012','DD/MM/YYYY'),'RUA X, 49', 5684.66);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12346,         1,         'MANUEL         DA         SILVA',
TO_DATE('05/10/1998','DD/MM/YYYY'),
     TO_DATE('10/11/2015','DD/MM/YYYY'),'RUA X, 31', 3542.11);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12347,         1,         'JANDIRA         DA         SILVA',
TO_DATE('10/12/2000','DD/MM/YYYY'),
     TO_DATE('15/09/2018','DD/MM/YYYY'),'RUA X, 25', 1875.96);

-- DEPTO = 2
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12348,         2,         'KATIA         REGINA         SOUZA',
TO_DATE('15/01/1995','DD/MM/YYYY'),
     TO_DATE('03/10/2015','DD/MM/YYYY'),'RUA Y, 49', 3894.63);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12349,         2,         'MARIA         DAS         DORES         SOUZA',
TO_DATE('18/08/1983','DD/MM/YYYY'),
    TO_DATE('23/10/2017','DD/MM/YYYY'),'RUA Y, 35', 1542.55);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12350,         2,         'ALFREDO         DE         SOUZA',
TO_DATE('04/05/1999','DD/MM/YYYY'),
    TO_DATE('05/06/2014','DD/MM/YYYY'),'RUA Y, 27', 5874.52);

-- DEPTO = 3
INSERT INTO T_SIP_FUNCIONARIO VALUES
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    (12351,        3,        'GISELE        DE        JESUS',
TO_DATE('15/04/1999','DD/MM/YYYY'),
    TO_DATE('20/03/2017','DD/MM/YYYY'),'RUA Z, 49', 1360.66);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12352,        3,        'RAFAEL        DE        JESUS',
TO_DATE('10/08/1998','DD/MM/YYYY'),
    TO_DATE('10/08/2012','DD/MM/YYYY'),'RUA Z, 55', 2563.44);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12353,        3,        'ROSANA        DE        JESUS',
TO_DATE('14/03/1987','DD/MM/YYYY'),
    TO_DATE('15/08/2019','DD/MM/YYYY'),'RUA Z, 79', 4879.55);

-- DEPTO = 4
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12354,       4,       'JOSEFINA       DE       ALMEIDA',
TO_DATE('16/10/1997','DD/MM/YYYY'),
    TO_DATE('25/03/2013','DD/MM/YYYY'),'RUA Y, 33', 4561.88);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12355,       4,       'LUCIANA       DE       ALMEIDA',
TO_DATE('10/02/1984','DD/MM/YYYY'),
    TO_DATE('28/09/2011','DD/MM/YYYY'),'RUA Y, 44', 2345.52);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12356,       4,       'THIAGO       DE       ALMEIDA',
TO_DATE('10/03/1998','DD/MM/YYYY'),
    TO_DATE('24/10/2018','DD/MM/YYYY'),'RUA Y, 55', 1254.22);

-- DEPTO = 5
INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12357,       5,       'LARISSSA       DE       CAMARGO',
TO_DATE('14/02/1997','DD/MM/YYYY'),
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    TO_DATE('04/08/2015','DD/MM/YYYY'),'RUA V, 22', 1245.55);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12358,        5,        'ANTONIO        DE        CAMARGO',
TO_DATE('25/01/1985','DD/MM/YYYY'),
    TO_DATE('12/08/2016','DD/MM/YYYY'),'RUA V, 44', 2451.33);

INSERT INTO T_SIP_FUNCIONARIO VALUES
    (12359,        5,        'JOSE        DE        CAMARGO',
TO_DATE('23/10/1998','DD/MM/YYYY'),
    TO_DATE('20/04/2017','DD/MM/YYYY'),'RUA V, 88', 6541.22);

-- POPULAR A TABELA DEPENDENTE. ESCOLHER 3 FUNCIONARIOS E
CADASTRAR 3 DEPENDENTES
-- PARA CADA FUNCIONARIO ESCOLHIDO
-- FUNCIONARIO = 12345
INSERT INTO T_SIP_DEPENDENTE VALUES
(12345, 1, 'JOANINHA', TO_DATE('12/10/2010','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12345, 2, 'JULINHA', TO_DATE('15/10/2012','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12345, 3, 'TONINHO', TO_DATE('22/10/2014','DD/MM/YYYY'));

-- FUNCIONARIO = 12356
INSERT INTO T_SIP_DEPENDENTE VALUES
(12356, 1,'JUNINHO', TO_DATE('02/10/2011','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12356, 2, 'ZEZINHO', TO_DATE('15/10/2013','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12356, 3, 'MARCELINHO', TO_DATE('17/10/2015','DD/MM/YYYY'));

-- FUNCIONARIO = 12359
INSERT INTO T_SIP_DEPENDENTE VALUES
(12359, 1,'MARIAZINHA', TO_DATE('11/06/2014','DD/MM/YYYY'));
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
INSERT INTO T_SIP_DEPENDENTE VALUES
(12359, 2,'LUIZINHA', TO_DATE('21/06/2016','DD/MM/YYYY'));
INSERT INTO T_SIP_DEPENDENTE VALUES
(12359, 3,'CARMINHA', TO_DATE('18/06/2018','DD/MM/YYYY'));

-- POPULAR A TABELA PROJETO COM 5 PROJETOS
-- 3 PROJETOS SEM A DATA TÉRMINO
INSERT INTO T_SIP_PROJETO VALUES
(10001,   'PROJETO   1',   TO_DATE('11/10/2016','DD/MM/YYYY'),
NULL);
INSERT INTO T_SIP_PROJETO VALUES
(10002,   'PROJETO   2',   TO_DATE('25/10/2013','DD/MM/YYYY'),
TO_DATE('15/04/2015','DD/MM/YYYY'));
INSERT INTO T_SIP_PROJETO VALUES
(10003,   'PROJETO   3',   TO_DATE('28/04/2012','DD/MM/YYYY'),
NULL);
INSERT INTO T_SIP_PROJETO VALUES
(10004,   'PROJETO   4',   TO_DATE('25/03/2017','DD/MM/YYYY'),
NULL);
INSERT INTO T_SIP_PROJETO VALUES
(10005,   'PROJETO   5',   TO_DATE('28/09/2014','DD/MM/YYYY'),
TO_DATE('15/09/2018','DD/MM/YYYY'));

-- POPULAR A TABELA IMPLANTACAO
-- PROJETO 1
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10001,    1,    12345,    TO_DATE('11/10/2016','DD/MM/YYYY'),
TO_DATE('15/09/2017','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10001, 2, 12346, TO_DATE('11/10/2016','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10001,    3,    12348,    TO_DATE('11/10/2016','DD/MM/YYYY'),
TO_DATE('25/06/2018','DD/MM/YYYY'));
-- PROJETO 2
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10002,    1,    12345,    TO_DATE('25/10/2013','DD/MM/YYYY'),
TO_DATE('15/04/2014','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10002,    2,    12352,    TO_DATE('25/10/2013','DD/MM/YYYY'),
TO_DATE('25/07/2014','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10002,    3,    12355,    TO_DATE('25/10/2013','DD/MM/YYYY'),
TO_DATE('15/04/2015','DD/MM/YYYY'));
-- PROJETO 3
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10003, 1, 12354, TO_DATE('28/04/2013','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10003, 2, 12355, TO_DATE('28/04/2013','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10003, 3, 12345, TO_DATE('28/04/2013','DD/MM/YYYY'), NULL);
-- PROJETO 4
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10004, 1, 12357, TO_DATE('25/03/2017','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10004, 2, 12358, TO_DATE('25/03/2017','DD/MM/YYYY'), NULL);
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10004,    3,    12354,    TO_DATE('25/03/2017','DD/MM/YYYY'),
TO_DATE('15/08/2018','DD/MM/YYYY'));
-- PROJETO 5
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10005,    1,    12345,    TO_DATE('28/09/2014','DD/MM/YYYY'),
TO_DATE('15/09/2018','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10005,    2,    12350,    TO_DATE('28/09/2014','DD/MM/YYYY'),
TO_DATE('15/09/2018','DD/MM/YYYY'));
INSERT INTO T_SIP_IMPLANTACAO VALUES
(10005,    3,    12354,    TO_DATE('28/09/2014','DD/MM/YYYY'),
TO_DATE('15/09/2018','DD/MM/YYYY'));
```

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- CONSULTAR TABELAS
SELECT * FROM T_SIP_DEPARTAMENTO;
SELECT * FROM T_SIP_FUNCIONARIO;
SELECT * FROM T_SIP_DEPENDENTE;
SELECT * FROM T_SIP_PROJETO;
SELECT * FROM T_SIP_IMPLANTACAO;

--EFETIVAR AS TRANSAÇÕES PENDENTES
COMMIT;
```

Código-fonte 3 - Script para popular as tabelas do banco de dados SIP
Fonte: Elaborado pela autora (2023)

1.3 Consulta Simples

Possibilita a **recuperação de todas as linhas e colunas de uma tabela**; a seleção de linhas que atendam aos **requisitos pré-estabelecidos**; e a definição das colunas, cujos dados serão selecionados.

1.3.1 Exemplo de comando SELECT em consultas simples

Exemplo do comando SELECT para recuperar registros ou linhas na tabela “FUNCIONARIO”. O comando SELECT retornará todas as colunas (*) e todas as linhas (não há condições) da tabela “FUNCIONARIO”.

```
-- CONSULTANDO TODAS AS LINHAS E COLUNAS DA TABELA FUNCIONARIO
SELECT * FROM T_SIP_FUNCIONARIO;
```

Código-fonte 4 - Exemplo de código-fonte SQL – comando SELECT
Fonte: Elaborado pela autora (2023)

Atenção:

Não é uma boa prática utilizar a cláusula SELECT *, pois, quando utilizamos, todas as colunas serão retornadas. É importante recuperar apenas as colunas que realmente serão necessárias, ou seja, que serão utilizadas nas funcionalidades que estão sendo desenvolvidas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Resultado da execução a consulta, através do comando “SELECT”:

	NR_MATRICULA	CD_DEPTO	NM_FUNCIONARIO	DT_NASCIMENTO	DT_ADMISSAO	DS_ENDERECO	VL_SALARIO_MENSAL
1	12345	1	JOAO DA SILVA	10/05/85	15/09/12	RUA X, 49	5684,66
2	12346	1	MANUEL DA SILVA	05/10/98	10/11/15	RUA X, 31	3542,11
3	12347	1	JANDIRA DA SILVA	10/12/00	15/09/18	RUA X, 25	1875,96
4	12348	2	KATIA REGINA SOUZA	15/01/95	03/10/15	RUA Y, 49	4159,46
5	12349	2	MARIA DAS DORES SOUZA	18/08/83	23/10/17	RUA Y, 35	1647,44
6	12350	2	ALFREDO DE SOUZA	04/05/99	05/06/14	RUA Y, 27	6273,99
7	12351	3	GISELE DE JESUS	15/04/99	20/03/17	RUA Z, 49	1360,66
8	12352	3	RAFAEL DE JESUS	10/08/98	10/08/12	RUA Z, 55	2563,44
9	12353	3	ROSANA DE JESUS	14/03/87	15/08/19	RUA Z, 79	4879,55
10	12354	4	JOSEFINA DE ALMEIDA	16/10/97	25/03/13	RUA Y, 33	4561,88
11	12355	4	LUCIANA DE ALMEIDA	10/02/84	28/09/11	RUA Y, 44	2345,52
12	12356	4	THIAGO DE ALMEIDA	10/03/98	24/10/18	RUA Y, 55	1254,22
13	12357	5	LARISSSA DE CAMARGO	14/02/97	04/08/15	RUA V, 22	1245,55
14	12358	5	ANTONIO DE CAMARGO	25/01/85	12/08/16	RUA V, 44	2451,33
15	12359	5	JOSE DE CAMARGO	23/10/98	20/04/17	RUA V, 88	6541,22

Figura 1 - Resultado da consulta à tabela FUNCIONARIO
Fonte: Elaborado pela autora (2023)

1.3.2 Exemplo de comando SELECT com declaração de colunas

Exemplo do comando SELECT para recuperar registros ou linhas na tabela “FUNCIONARIO”. Neste exemplo, as colunas a serem retornadas são declaradas na cláusula SELECT. O comando SELECT retornará apenas às colunas declaradas e todas as linhas (não há condições) da tabela “FUNCIONARIO”.

```
-- Especificando as colunas que deseja consultar e recuperando
-- todas as linhas da tabela FUNCIONARIO
SELECT NR_MATRICULA, NM_FUNCIONARIO, VL_SALARIO_MENSAL
  FROM T_SIP_FUNCIONARIO;
```

Código-fonte 5 - Exemplo de código-fonte SQL – comando SELECT
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	NR_MATRICULA	NM_FUNCIONARIO	VL_SALARIO_MENSAL
1	12345	JOAO DA SILVA	5684,66
2	12346	MANUEL DA SILVA	3542,11
3	12347	JANDIRA DA SILVA	1875,96
4	12348	KATIA REGINA SOUZA	4159,46
5	12349	MARIA DAS DORES SOUZA	1647,44
6	12350	ALFREDO DE SOUZA	6273,99
7	12351	GISELE DE JESUS	1360,66
8	12352	RAFAEL DE JESUS	2563,44
9	12353	ROSANA DE JESUS	4879,55
10	12354	JOSEFINA DE ALMEIDA	4561,88
11	12355	LUCIANA DE ALMEIDA	2345,52
12	12356	THIAGO DE ALMEIDA	1254,22
13	12357	LARISSSA DE CAMARGO	1245,55
14	12358	ANTONIO DE CAMARGO	2451,33
15	12359	JOSE DE CAMARGO	6541,22

Figura 2 - Resultado da consulta à tabela FUNCIONARIO
Fonte: Elaborado pela autora (2023)

1.3.3 Exemplo de comando SELECT com declaração de colunas e condição (filtro)

Para entender melhor, iremos analisar um exemplo do comando SELECT para recuperar registros ou linhas na tabela “FUNCIONARIO”. Neste exemplo, as colunas a serem retornadas são declaradas na cláusula SELECT. Também verificaremos a existência de uma condição. A condição realizará um filtro e recuperará apenas as linhas que satisfazem a condição especificada.

O comando SELECT retornará apenas às colunas declaradas e as linhas que atendem a condição especificada (filtro) da tabela “FUNCIONARIO”.

```
-- Especificando as colunas que deseja consultar e filtrando -
-- as linhas da tabela FUNCIONARIO.
SELECT NR_MATRICULA, CD_DEPTO,
       NM_FUNCIONARIO, VL_SALARIO_MENSAL
  FROM T_SIP_FUNCIONARIO
 WHERE CD_DEPTO = 3;
```

Código-fonte 6 - Exemplo de código-fonte SQL – comando SELECT com WHERE
Fonte: Elaborado pela autora (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Resultado da execução a consulta, através do comando “SELECT”:

	NR_MATRICULA	CD_DEPTO	NM_FUNCIONARIO	VL_SALARIO_MENSAL
1	12351	3	GISELE DE JESUS	1360,66
2	12352	3	RAFAEL DE JESUS	2563,44
3	12353	3	ROSANA DE JESUS	4879,55

Figura 3 - Resultado da consulta à tabela FUNCIONARIO (Select/Where)
Fonte: Elaborado pela autora (2023)

1.3.4 Exemplo de comando SELECT com a opção DISTINCT

Agora, iremos analisar um exemplo do comando SELECT para recuperar registros ou linhas na tabela “FUNCIONARIO”. Neste exemplo, as colunas a serem retornadas são declaradas na cláusula SELECT.

A instrução SELECT DISTINCT é utilizada para retornar valores distintos (únicos), ou seja, valores diferentes, sem duplicá-los.

O comando SELECT retornará apenas as colunas declaradas com valores distintos (únicos), e todas as linhas da tabela “FUNCIONARIO”.

Exemplificaremos a situação em que é necessário recuperar todos os departamentos que possuem funcionários.

```
-- RECUPERANDO OS DEPARTAMENTOS (SEM REPETIÇÃO) QUE POSSUEM
-- FUNCIONARIOS
SELECT DISTINCT CD_DEPTO FROM T_SIP_FUNCIONARIO;
-- OU
SELECT UNIQUE CD_DEPTO FROM T_SIP_FUNCIONARIO;
```

Código-fonte 7 - Exemplo de código-fonte SQL – comando SELECT, com DISTINCT
Fonte: Elaborado pela autora (2023)

Dentro de uma tabela, uma coluna geralmente contém muitos valores duplicados. Muitas vezes, precisamos listar apenas os valores diferentes (distintos).

Resultado da execução a consulta, através do comando “SELECT”:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	CD_DEPTO
1	2
2	4
3	3
4	5
5	1

Figura 4 - Resultado da consulta à tabela FUNCIONARIO (Select/Distinct)
Fonte: Elaborado pela autora (2023)

1.4 Operadores Aritméticos

Os operadores aritméticos são utilizados nas instruções SQL.

Sempre que houver mais de um operador, a precedência matemática será respeitada, ou seja, a multiplicação e a divisão serão avaliadas em primeiro lugar, e o uso dos parênteses pode alterar a ordem de execução dos operadores.

OPERADOR	DESCRIÇÃO
+	Adição
-	Subtração
*	Multiplicação
/	Divisão

Quadro 1 - Operadores Aritméticos
Fonte: Elaborado pela autora (2023)

Exemplos de comando SELECT com operadores aritméticos:

O comando SELECT retornará apenas as colunas declaradas da tabela “FUNCIONARIO”.

Observe que estamos realizando um cálculo durante a declaração das colunas no comando SELECT, portanto serão recuperados todos os funcionários cadastrados e ainda será realizado o cálculo de 5% de aumento do salário mensal.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- EXEMPLO DE CONSULTA UTILIZANDO OPERADORES ARITMÉTICOS
SELECT NR_MATRICULA, NM_FUNCIONARIO,
       VL_SALARIO_MENSAL,
       VL_SALARIO_MENSAL * 1.05
  FROM T_SIP_FUNCIONARIO;
```

Código-fonte 8 - Exemplo de código-fonte SQL – comando SELECT com operador aritmético
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

	NR_MATRICULA	NM_FUNCIONARIO	VL_SALARIO_MENSAL	VL_SALARIO_MENSAL*1.05
1	12345	JOAO DA SILVA	5684,66	5968,893
2	12346	MANUEL DA SILVA	3542,11	3719,2155
3	12347	JANDIRA DA SILVA	1875,96	1969,758
4	12348	KATIA REGINA SOUZA	4159,46	4367,433
5	12349	MARIA DAS DORES SOUZA	1647,44	1729,812
6	12350	ALFREDO DE SOUZA	6273,99	6587,6895
7	12351	GISELE DE JESUS	1360,66	1428,693
8	12352	RAFAEL DE JESUS	2563,44	2691,612
9	12353	ROSANA DE JESUS	4879,55	5123,5275
10	12354	JOSEFINA DE ALMEIDA	4561,88	4789,974
11	12355	LUCIANA DE ALMEIDA	2345,52	2462,796
12	12356	THIAGO DE ALMEIDA	1254,22	1316,931
13	12357	LARISSSA DE CAMARGO	1245,55	1307,8275
14	12358	ANTONIO DE CAMARGO	2451,33	2573,8965
15	12359	JOSE DE CAMARGO	6541,22	6868,281

Figura 5 - Resultado da consulta à tabela FUNCIONARIO, comando SELECT com operador aritmético
Fonte: Elaborado pela autora (2023)

1.5 ALIAS – Apelido para colunas

Às vezes, ao recuperar dados através de um comando SELECT, o nome da coluna (campo) não é muito claro ou apropriado para a apresentação dos dados.

Os apelidos são úteis para melhorar a legibilidade do resultado da consulta, facilitando o entendimento por parte do usuário, portanto, é possível renomear o cabeçalho (rótulo) da coluna que será recuperada.

Por isso, podemos utilizar um “apelido” para a coluna, que chamamos de “ALIAS”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

É muito útil quando queremos retornar um nome adequado para o resultado de um cálculo, por exemplo.

Deve ser escrito após o nome da coluna ou cálculo, se for uma palavra composta, caso tenha caracteres especiais (acentos, por exemplo) ou que faça distinção entre maiúsculas e minúsculas. Além disso, deve ser escrito entre aspas dupla. É importante ressaltar que não deve ser uma palavra reservada (SELECT, WHERE etc.).

Podemos utilizar o “AS” (opcional) para explicitar o apelido de uma coluna.

Exemplo de consulta utilizando ALIAS para colunas.

```
-- EXEMPLOS DE APELIDOS DE COLUNAS
SELECT NR_MATRICULA,
       NM_FUNCIONARIO FUNCIONARIO,
       VL_SALARIO_MENSAL "SALÁRIO MENSAL",
       VL_SALARIO_MENSAL * 1.05 AS "SALÁRIO REAJUSTADO"
  FROM T_SIP_FUNCIONARIO;
```

Código-fonte 9 - Exemplo de código-fonte SQL – comando SELECT com ALIAS de colunas
Fonte: Elaborado pela autora (2023)

Observe o cabeçalho das colunas, as colunas recuperadas são exibidas com o apelido especificado no momento da consulta.

Resultado da execução a consulta, através do comando “SELECT”:

	NR_MATRICULA	FUNCIONARIO	SALÁRIO MENSAL	SALÁRIO REAJUSTADO
1	12345	JOAO DA SILVA	5684,66	5968,893
2	12346	MANUEL DA SILVA	3542,11	3719,2155
3	12347	JANDIRA DA SILVA	1875,96	1969,758
4	12348	KATIA REGINA SOUZA	4159,46	4367,433
5	12349	MARIA DAS DORES SOUZA	1647,44	1729,812
6	12350	ALFREDO DE SOUZA	6273,99	6587,6895
7	12351	GISELE DE JESUS	1360,66	1428,693
8	12352	RAFAEL DE JESUS	2563,44	2691,612
9	12353	ROSANA DE JESUS	4879,55	5123,5275
10	12354	JOSEFINA DE ALMEIDA	4561,88	4789,974
11	12355	LUCIANA DE ALMEIDA	2345,52	2462,796
12	12356	THIAGO DE ALMEIDA	1254,22	1316,931
13	12357	LARISSSA DE CAMARGO	1245,55	1307,8275
14	12358	ANTONIO DE CAMARGO	2451,33	2573,8965
15	12359	JOSE DE CAMARGO	6541,22	6868,281

Figura 6 - Resultado da consulta à tabela FUNCIONARIO com ALIAS de colunas
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.6 Cláusula ORDER BY

O resultado de uma consulta pode ser apresentado de forma ordenada (classificada), utilizando a cláusula ORDER BY.

A cláusula ORDER BY é utilizada para classificar (ordenar) as linhas recuperadas por uma consulta. Podemos especificar uma ou mais colunas para classificar (ordenar) os dados de uma tabela.

Essa cláusula é declarada no final de um comando SELECT e pode ordenar o resultado em ordem ascendente (“ASC”, que é a opção “DEFAULT”) e descendente (“DESC”).

A cláusula ORDER BY aceita números representando a coluna na ordem em que foi declarada na cláusula SELECT, além de aceitar o ALIAS (apelido), caso tenha sido dado a alguma coluna.

O comando SELECT retornará apenas as colunas declaradas e todas as linhas da tabela “DEPARTAMENTO”. As linhas serão retornadas conforme a classificação fornecida na instrução SQL.

Neste exemplo, serão recuperados todos os departamentos cadastrados e ordenados de forma ascendente (crescente), por nome do departamento.

```
-- APRESENTANDO RESULTADOS DE FORMA ORDENADA
-- ORDER BY ASC
  SELECT CD_DEPTO, NM_DEPTO
    FROM T_SIP_DEPARTAMENTO
ORDER BY NM_DEPTO;
```

Código-fonte 10 - Exemplo de código-fonte SQL – comando SELECT com ORDER BY
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

	CD_DEPTO	NM_DEPTO
1	3	CONTAS A PAGAR
2	4	FATURAMENTO
3	1	FINANCEIRO
4	5	RECURSOS HUMANOS
5	2	TECNOLOGIA DA INFORMAÇÃO

Figura 7 - Resultado da consulta à tabela DEPARTAMENTO – ORDER BY
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O comando SELECT retornará apenas as colunas declaradas e todas as linhas da tabela “DEPARTAMENTO”. As linhas serão retornadas conforme a classificação fornecida na instrução SQL.

Neste exemplo, serão recuperados todos os departamentos cadastrados e ordenados de forma descendente (decrescente), por nome do departamento.

```
-- APRESENTANDO RESULTADOS DE FORMA ORDENADA
-- ORDER BY DESC
  SELECT CD_DEPTO, NM_DEPTO
    FROM T_SIP_DEPARTAMENTO
ORDER BY NM_DEPTO DESC;
```

Código-fonte 11 - Exemplo de código-fonte SQL – comando SELECT com ORDER BY
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

	CD_DEPTO	NM_DEPTO
1	2	TECNOLOGIA DA INFORMAÇÃO
2	5	RECURSOS HUMANOS
3	1	FINANCEIRO
4	4	FATURAMENTO
5	3	CONTAS A PAGAR

Figura 8 - Resultado da consulta à tabela DEPARTAMENTO – ORDER BY
Fonte: Elaborado pela autora (2023)

O comando SELECT retornará apenas as colunas declaradas e todas as linhas da tabela “FUNCIONARIO”. As linhas serão retornadas conforme a classificação fornecida na instrução SQL.

Neste exemplo, serão recuperados todos os funcionários cadastrados, ordenados de forma ascendente (crescente), por código do departamento, e ordenados de forma descendente (decrescente) por salário mensal.

Nesse exemplo, substituímos o nome das colunas declaradas na cláusula ORDER BY pelo número de ordem das colunas declaradas na cláusula SELECT.

A classificação é feita conforme a ordem das colunas especificadas na cláusula ORDER BY.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- APRESENTANDO RESULTADOS DE FORMA ORDENADA
-- ORDER BY UTILIZANDO NÚMEROS QUE REPRESENTAM
-- A POSIÇÃO DAS COLUNAS NO SELECT
  SELECT NR_MATRICULA, CD_DEPTO,
         NM_FUNCIONARIO, VL_SALARIO_MENSAL
    FROM T_SIP_FUNCIONARIO
ORDER BY 2 ASC, 4 DESC;
```

Código-fonte 12 - Exemplo de código-fonte SQL – comando SELECT com ORDER BY
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

	NR_MATRICULA	CD_DEPTO	NM_FUNCIONARIO	VL_SALARIO_MENSAL
1	12345	1	JOAO DA SILVA	5684,66
2	12346	1	MANUEL DA SILVA	3542,11
3	12347	1	JANDIRA DA SILVA	1875,96
4	12350	2	ALFREDO DE SOUZA	6273,99
5	12348	2	KATIA REGINA SOUZA	4159,46
6	12349	2	MARIA DAS DORES SOUZA	1647,44
7	12353	3	ROSANA DE JESUS	4879,55
8	12352	3	RAFAEL DE JESUS	2563,44
9	12351	3	GISELE DE JESUS	1360,66
10	12354	4	JOSEFINA DE ALMEIDA	4561,88
11	12355	4	LUCIANA DE ALMEIDA	2345,52
12	12356	4	THIAGO DE ALMEIDA	1254,22
13	12359	5	JOSE DE CAMARGO	6541,22
14	12358	5	ANTONIO DE CAMARGO	2451,33
15	12357	5	LARISSSA DE CAMARGO	1245,55

Figura 9 - Resultado da consulta à tabela FUNCIONARIO – ORDER BY
Fonte: Elaborado pela autora (2023)

O comando SELECT retornará apenas as colunas declaradas e todas as linhas da tabela “FUNCIONARIO”. As linhas serão retornadas conforme a classificação fornecida na instrução SQL.

Neste exemplo, serão recuperados todos os funcionários cadastrados, será realizado o cálculo de reajuste do salário em 5% e será apresentado o resultado ordenado de forma descendente (decrescente) por salário reajustado.

Neste caso iremos utilizar o apelido dado ao resultado do cálculo na cláusula ORDER BY.

A classificação é realizada conforme a ordem das colunas especificadas na cláusula ORDER BY.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- APRESENTANDO RESULTADOS DE FORMA ORDENADA
-- ORDER BY UTILIZANDO O APELIDO DAS COLUNAS
  SELECT NR_MATRICULA          "MATRICULA",
         NM_FUNCIONARIO        "NOME ",
         VL_SALARIO_MENSAL     "SALÁRIO MENSAL",
         VL_SALARIO_MENSAL * 1.05 "SALÁRIO REAJUSTADO"
    FROM T_SIP_FUNCIONARIO
ORDER BY "SALÁRIO REAJUSTADO" DESC;
```

Código-fonte 13 - Exemplo de código-fonte SQL – comando SELECT com ORDER BY
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

	MATRICULA	NOME	SALÁRIO MENSAL	SALÁRIO REAJUSTADO
1	12359	JOSE DE CAMARGO	6541,22	6868,281
2	12350	ALFREDO DE SOUZA	6273,99	6587,6895
3	12345	JOAO DA SILVA	5684,66	5968,893
4	12353	ROSANA DE JESUS	4879,55	5123,5275
5	12354	JOSEFINA DE ALMEIDA	4561,88	4789,974
6	12348	KATIA REGINA SOUZA	4159,46	4367,433
7	12346	MANUEL DA SILVA	3542,11	3719,2155
8	12352	RAFAEL DE JESUS	2563,44	2691,612
9	12358	ANTONIO DE CAMARGO	2451,33	2573,8965
10	12355	LUCIANA DE ALMEIDA	2345,52	2462,796
11	12347	JANDIRA DA SILVA	1875,96	1969,758
12	12349	MARIA DAS DORES SOUZA	1647,44	1729,812
13	12351	GISELE DE JESUS	1360,66	1428,693
14	12356	THIAGO DE ALMEIDA	1254,22	1316,931
15	12357	LARISSSA DE CAMARGO	1245,55	1307,8275

Figura 10 - Resultado da consulta à tabela FUNCIONARIO – ORDER BY
Fonte: Elaborado pela autora (2023)

1.7 Consulta em tabelas utilizando filtros

Quando inserimos filtros na consulta das tabelas, utilizamos operadores para criar esses filtros, que funcionam como condições que precisam ser satisfeitas, para que os dados sejam recuperados.

Os operadores podem ser utilizados para estabelecer uma relação de comparação entre valores ou expressões, resultando sempre em um valor lógico (booleano), sendo verdadeiro ou falso.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Os operadores podem ser utilizados em consultas, atualizações ou exclusões de dados.

1.7.1 Operadores relacionais

Os operadores relacionais são utilizados para realizar comparações entre valores em estruturas de controle.

São operadores relacionais na linguagem SQL:

OPERADOR	SIGNIFICADO	EXEMPLO
=	Igual	codigo=2
<	Menor que	preco<10
<=	Menor ou igual a	preco<=10
>	Maior que	preco>10
>=	Maior ou igual a	preco>=10
<>Ou !=	Diferente	codigo<>2

Figura 11 - Operadores Relacionais
Fonte: Elaborado pela autora (2017)

Exemplos de comando SELECT com operadores relacionais:

O comando SELECT retornará apenas as colunas declaradas e as linhas que atendem a condição especificada (filtro) da tabela “FUNCIONARIO”.

Observe que no filtro utilizamos o operador relacional (> maior). Neste exemplo, em função do filtro, serão recuperados todos os funcionários com salários superiores a R$ 1.500,50.

Importante ressaltar que utilizamos o ponto decimal, informando o valor decimal que será utilizado durante a consulta.

```
-- FILTRANDO POR NÚMEROS
SELECT NR_MATRICULA, NM_FUNCIONARIO, VL_SALARIO_MENSAL
  FROM T_SIP_FUNCIONARIO
 WHERE VL_SALARIO_MENSAL > 1500.50;
```

Código-fonte 14 - Exemplo de código-fonte SQL – comando SELECT com operadores relacionais
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Resultado da execução a consulta por meio do comando “SELECT”:

	NR_MATRICULA	NM_FUNCIONARIO	VL_SALARIO_MENSAL
1	12345	JOAO DA SILVA	5684,66
2	12346	MANUEL DA SILVA	3542,11
3	12347	JANDIRA DA SILVA	1875,96
4	12348	KATIA REGINA SOUZA	4159,46
5	12349	MARIA DAS DORES SOUZA	1647,44
6	12350	ALFREDO DE SOUZA	6273,99
7	12352	RAFAEL DE JESUS	2563,44
8	12353	ROSANA DE JESUS	4879,55
9	12354	JOSEFINA DE ALMEIDA	4561,88
10	12355	LUCIANA DE ALMEIDA	2345,52
11	12358	ANTONIO DE CAMARGO	2451,33
12	12359	JOSE DE CAMARGO	6541,22

Figura 12 - Resultado da consulta à tabela FUNCIONARIO – Operador Relacional
Fonte: Elaborado pela autora (2023)

O comando SELECT retornará apenas as colunas declaradas e as linhas que atendem a condição especificada (filtro) da tabela “FUNCIONARIO”.

Observe que no filtro utilizamos o operador relacional (= igual). Neste exemplo, em função do filtro, serão recuperados todos os funcionários com nome exatamente igual ‘JOÃO DA SILVA’.

Importante ressaltar que devemos informar o conteúdo texto (cadeia de caracteres) a ser avaliado entre aspas simples.

```
-- FILTRANTO POR TEXTO (CADEIA DE CARACTERES)
SELECT NR_MATRICULA, NM_FUNCIONARIO
  FROM T_SIP_FUNCIONARIO
 WHERE NM_FUNCIONARIO = 'JOAO DA SILVA';
```

Código-fonte 15 - Exemplo de código-fonte SQL – comando SELECT com operadores relacionais
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

	NR_MATRICULA	NM_FUNCIONARIO
1	12345	JOAO DA SILVA

Figura 13 - Resultado da consulta à tabela FUNCIONARIO – Operador Relacional
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

É importante observar que o conteúdo armazenado nas tabelas é “CASE SENSITIVE”, ou seja, temos a diferenciação entre letras maiúsculas e minúsculas. Para minimizar isto durante a consulta e filtro por colunas que armazenam texto (cadeia de caracteres), utilizamos no momento da criação do filtro as funções “`UPPER(<conteúdo>)`”, que converte um conteúdo para letras maiúsculas, ou a “`LOWER(<conteúdo >)`”, que converte um conteúdo para letras minúsculas.

Com o uso de uma das funções na criação do filtro, garantimos que todas as ocorrências encontradas serão recuperadas independentemente da forma que foram escritas (maiúsculas ou minúsculas).

Exemplificaremos a consulta com o filtro apresentado anteriormente, utilizando as funções “`UPPER(<conteúdo >)`” e “`LOWER(<conteúdo >)`”.

```
-- UTILIZANDO A CONVERSÃO DE TEXTO
SELECT NR_MATRICULA, NM_FUNCIONARIO
  FROM T_SIP_FUNCIONARIO
 WHERE UPPER(NM_FUNCIONARIO) = 'JOAO DA SILVA';
-- OU
 SELECT NR_MATRICULA, NM_FUNCIONARIO
  FROM T_SIP_FUNCIONARIO
 WHERE LOWER(NM_FUNCIONARIO) = 'joao da silva';
```

Código-fonte 16 - Exemplo de código-fonte SQL – comando SELECT com operadores relacionais
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

	NR_MATRICULA	NM_FUNCIONARIO
1	12345	JOAO DA SILVA

Figura 14 - Resultado da consulta à tabela FUNCIONARIO – Operador Relacional
Fonte: Elaborado pela autora (2023)

O comando SELECT retornará apenas as colunas declaradas e as linhas que atendem a condição especificada (filtro) da tabela “FUNCIONARIO”.

Observe que no filtro utilizamos o operador relacional (>= maior igual). Neste exemplo, em função do filtro, serão recuperados todos os funcionários admitidos a partir de 01/08/2015 (inclusive).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Além disso, é importante destacar que devemos utilizar a função "`TO_DATE(<data>,<formato>)`" no conteúdo data/hora a ser avaliado.

```
-- FILTRANDO POR DATA
SELECT NR_MATRICULA, NM_FUNCIONARIO, DT_ADMISSAO
  FROM T_SIP_FUNCIONARIO
 WHERE DT_ADMISSAO >= TO_DATE('01/08/2015','DD/MM/YYYY');
```

Código-fonte 17 - Exemplo de código-fonte SQL – comando SELECT com operadores relacionais
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando "SELECT":

	NR_MATRICULA	NM_FUNCIONARIO	DT_ADMISSAO
1	12346	MANUEL DA SILVA	10/11/15
2	12347	JANDIRA DA SILVA	15/09/18
3	12348	KATIA REGINA SOUZA	03/10/15
4	12349	MARIA DAS DORES SOUZA	23/10/17
5	12351	GISELE DE JESUS	20/03/17
6	12353	ROSANA DE JESUS	15/08/19
7	12356	THIAGO DE ALMEIDA	24/10/18
8	12357	LARISSSA DE CAMARGO	04/08/15
9	12358	ANTONIO DE CAMARGO	12/08/16
10	12359	JOSE DE CAMARGO	20/04/17

Figura 15 - Resultado da consulta à tabela FUNCIONARIO – Operador Relacional
Fonte: Elaborado pela autora (2023)

1.7.2 Operadores lógicos

Os operadores lógicos são utilizados para filtrar linhas de uma base de dados, utilizando mais de uma condição.

- O operador AND recupera uma linha, se todas as condições separadas por AND forem verdadeiras.
- O operador OR recupera uma linha, se qualquer uma das condições separadas por OR forem verdadeiras.
- O operador NOT recupera uma linha, se a condição (s) não for verdadeira.

São operadores lógicos, na linguagem SQL:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

OPERADOR	DESCRIÇÃO
AND	Retorna TRUE se ambas condições forem verdadeiras
OR	Retorna TRUE se ao menos uma das condições for verdadeira
NOT	Retorna TRUE se a condição for falsa

Figura 16 - Operadores LÓGICOS
Fonte: Elaborado pela autora (2017)

Exemplos de comando SELECT com operadores lógicos:

O comando SELECT retornará apenas as colunas declaradas e as linhas que atendem a condição especificada (filtro) da tabela “FUNCIONARIO”.

Observe que no filtro utilizamos os operadores relacionais (>= maior igual e = igual) e o operador lógico AND. Neste exemplo, em função do filtro, serão recuperados todos os funcionários alocados no departamento de código igual a três e que foram admitidos a partir de 01/08/2015 (inclusive).

É importante ressaltar que devemos utilizar a função “`TO_DATE(<data>,<formato>)`” no conteúdo data/hora a ser avaliado.

```
-- OPERADOR LOGICO AND -> PRECISA ATENDER AS DUAS CONDIÇÕES
-- PARA RECUPERAR OS VALORES
SELECT NR_MATRICULA, NM_FUNCIONARIO, CD_DEPTO, DT_ADMISSAO
  FROM T_SIP_FUNCIONARIO
 WHERE DT_ADMISSAO >= TO_DATE('01/08/2015','DD/MM/YYYY')
            AND CD_DEPTO = 3;
```

Código-fonte 18 - Exemplo de código-fonte SQL – comando SELECT com operadores lógicos
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

	NR_MATRICULA	NM_FUNCIONARIO	CD_DEPTO	DT_ADMISSAO
1	12351	GISELE DE JESUS	3	20/03/17
2	12353	ROSANA DE JESUS	3	15/08/19

Figura 17 - Resultado da consulta à tabela FUNCIONARIO – Operador Lógico
Fonte: Elaborado pela autora (2023)

O comando SELECT retornará apenas as colunas declaradas e as linhas que atendem a condição especificada (filtro) da tabela “FUNCIONARIO”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Observe que no filtro utilizamos os operadores relacionais (> maior e = igual) e o operador lógico OR. Neste exemplo, em função do filtro, serão recuperados todos os funcionários alocados no departamento de código igual a quatro ou que possuem salário maior que R$ 3.000,00.

```
-- OPERADOR LOGICO OR -> PRECISA ANTENDER QUALQUER UMA
-- DAS CONDIÇÕES PARA RECUPERAR OS VALORES
SELECT NR_MATRICULA, NM_FUNCIONARIO,
       CD_DEPTO, VL_SALARIO_MENSAL
  FROM T_SIP_FUNCIONARIO
 WHERE CD_DEPTO = 4
       OR VL_SALARIO_MENSAL > 3000;
```

Código-fonte 19 - Exemplo de código-fonte SQL – comando SELECT com operadores lógicos
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

	NR_MATRICULA	NM_FUNCIONARIO	CD_DEPTO	VL_SALARIO_MENSAL
1	12345	JOAO DA SILVA	1	5684,66
2	12346	MANUEL DA SILVA	1	3542,11
3	12348	KATIA REGINA SOUZA	2	4159,46
4	12350	ALFREDO DE SOUZA	2	6273,99
5	12353	ROSANA DE JESUS	3	4879,55
6	12354	JOSEFINA DE ALMEIDA	4	4561,88
7	12355	LUCIANA DE ALMEIDA	4	2345,52
8	12356	THIAGO DE ALMEIDA	4	1254,22
9	12359	JOSE DE CAMARGO	5	6541,22

Figura 18 - Resultado da consulta à tabela FUNCIONARIO – Operador Lógico
Fonte: Elaborado pela autora (2023)

O comando SELECT retornará apenas as colunas declaradas e as linhas que atendem a condição especificada (filtro) da tabela “FUNCIONARIO”.

Observe que no filtro utilizamos o operador relacional (= igual) e o operador lógico NOT. Neste exemplo, em função do filtro, serão recuperados todos os funcionários que não estão alocados no departamento de código igual a quatro.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- OPERADOR NOT
SELECT NR_MATRICULA, NM_FUNCIONARIO,
       CD_DEPTO, VL_SALARIO_MENSAL
  FROM T_SIP_FUNCIONARIO
 WHERE NOT CD_DEPTO = 4;
```

Código-fonte 20 - Exemplo de código-fonte SQL – comando SELECT com operadores lógicos
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

	NR_MATRICULA	NM_FUNCIONARIO	CD_DEPTO	VL_SALARIO_MENSAL
1	12345	JOAO DA SILVA	1	5684,66
2	12346	MANUEL DA SILVA	1	3542,11
3	12347	JANDIRA DA SILVA	1	1875,96
4	12348	KATIA REGINA SOUZA	2	4159,46
5	12349	MARIA DAS DORES SOUZA	2	1647,44
6	12350	ALFREDO DE SOUZA	2	6273,99
7	12351	GISELE DE JESUS	3	1360,66
8	12352	RAFAEL DE JESUS	3	2563,44
9	12353	ROSANA DE JESUS	3	4879,55
10	12357	LARISSSA DE CAMARGO	5	1245,55
11	12358	ANTONIO DE CAMARGO	5	2451,33
12	12359	JOSE DE CAMARGO	5	6541,22

Figura 19 - Resultado da consulta à tabela FUNCIONARIO – Operador Lógico
Fonte: Elaborado pela autora (2023)

1.7.3 Operadores Especiais ou operadores SQL

Os operadores SQL permitem limitar as linhas recuperadas com base na correspondência de padrão de *strings*, listas de valores, intervalos de valores e valores nulos.

São operadores especiais, na linguagem SQL:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

OPERADOR	DESCRIÇÃO
BETWEEN... AND...	Entre dois valores (inclusive)
IN (*lista*)	Compara o valor de uma coluna com um conjunto de valor
Like *valor*	Compara cadeia de caracteres
IS NULL/ IS NOT NULL	É um valor nulo. Não nulo

Figura 20 - Operadores Especiais ou Operadores SQL
Fonte: Elaborado pela autora (2017)

Exemplos de comando SELECT com operadores especiais:

- **Operador SQL BETWEEN:**

O operador especial ou SQL BETWEEN é utilizado para recuperar linhas, cujo valor da coluna está em um intervalo especificado. O intervalo inclui os valores das duas extremidades do intervalo.

O comando SELECT retornará apenas as colunas declaradas e as linhas que atendem a condição especificada (filtro) da tabela “FUNCIONARIO”.

Observe que no filtro utilizamos o operador especial BETWEEN. Neste exemplo, em função do filtro, serão recuperados todos os funcionários com remuneração (salário) entre R$ 1000.00 (inclusive) e R$ 2500.00 (inclusive).

```
-- OPERADOR BETWEEN
SELECT NR_MATRICULA, NM_FUNCIONARIO,
       CD_DEPTO, VL_SALARIO_MENSAL
FROM   T_SIP_FUNCIONARIO
WHERE  VL_SALARIO_MENSAL BETWEEN 1000 AND 2500;
```

Código-fonte 21 - Exemplo de código-fonte SQL – comando SELECT com operador BETWEEN
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	NR_MATRICULA	NM_FUNCIONARIO	CD_DEPTO	VL_SALARIO_MENSAL
1	12347	JANDIRA DA SILVA	1	1875,96
2	12349	MARIA DAS DORES SOUZA	2	1647,44
3	12351	GISELE DE JESUS	3	1360,66
4	12355	LUCIANA DE ALMEIDA	4	2345,52
5	12356	THIAGO DE ALMEIDA	4	1254,22
6	12357	LARISSSA DE CAMARGO	5	1245,55
7	12358	ANTONIO DE CAMARGO	5	2451,33

Figura 21 - Resultado da consulta à tabela FUNCIONARIO – Operador Between
Fonte: Elaborado pela autora (2023)

O comando SELECT retornará apenas as colunas declaradas e as linhas que atendem a condição especificada (filtro) da tabela “FUNCIONARIO”.

Observe que no filtro utilizamos o operador especial BETWEEN. Neste exemplo, em função do filtro, serão recuperados todos os funcionários admitidos entre “01/01/2013” (inclusive) e “30/12/2015” (inclusive).

```
-- OPERADOR BETWEEN
SELECT NR_MATRICULA, NM_FUNCIONARIO,
       CD_DEPTO, DT_ADMISSAO
FROM   T_SIP_FUNCIONARIO
WHERE  DT_ADMISSAO BETWEEN TO_DATE('01/01/2013','DD/MM/YYYY')
       AND  TO_DATE('30/12/2015','DD/MM/YYYY');
```

Código-fonte 22 - Exemplo de código-fonte SQL – comando SELECT com operador BETWEEN
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

	NR_MATRICULA	NM_FUNCIONARIO	CD_DEPTO	DT_ADMISSAO
1	12346	MANUEL DA SILVA	1	10/11/15
2	12348	KATIA REGINA SOUZA	2	03/10/15
3	12350	ALFREDO DE SOUZA	2	05/06/14
4	12354	JOSEFINA DE ALMEIDA	4	25/03/13
5	12357	LARISSSA DE CAMARGO	5	04/08/15

Figura 22 - Resultado da consulta à tabela FUNCIONARIO – Operador Between
Fonte: Elaborado pela autora (2023)

- **Operador SQL IN():**

O operador especial ou SQL IN() é utilizado para recuperar linhas, cujo valor da coluna está em uma lista de valores. O comando SELECT retornará apenas as

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

colunas declaradas e as linhas que atendem a condição especificada (filtro) da tabela “FUNCIONARIO”.

Observe que no filtro utilizamos o operador especial IN(). Neste exemplo, em função do filtro, serão recuperados todos os funcionários alocados nos departamentos um, dois ou três.

```
-- OPERADOR IN
SELECT NR_MATRICULA, NM_FUNCIONARIO, CD_DEPTO
  FROM   T_SIP_FUNCIONARIO
 WHERE  CD_DEPTO IN (1,2,3);
```

Código-fonte 23 - Exemplo de código-fonte SQL – Comando SELECT com operador IN()
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

	NR_MATRICULA	NM_FUNCIONARIO	CD_DEPTO
1	12345	JOAO DA SILVA	1
2	12346	MANUEL DA SILVA	1
3	12347	JANDIRA DA SILVA	1
4	12348	KATIA REGINA SOUZA	2
5	12349	MARIA DAS DORES SOUZA	2
6	12350	ALFREDO DE SOUZA	2
7	12351	GISELE DE JESUS	3
8	12352	RAFAEL DE JESUS	3
9	12353	ROSANA DE JESUS	3

Figura 23 - Resultado da consulta à tabela FUNCIONARIO – Operador IN()
Fonte: Elaborado pela autora (2023)

O operador especial ou SQL IN() é utilizado para recuperar linhas, cujo valor da coluna está em uma lista de valores. Podemos associar o operador lógico NOT ao operador especial IN(), criando uma negação do que é informado na lista de valores. O comando SELECT retornará apenas as colunas declaradas e as linhas que atendem a condição especificada (filtro) da tabela “FUNCIONARIO”.

Observe que no filtro utilizamos o operador especial IN() associado ao operador lógico NOT. Neste exemplo, em função do filtro, serão recuperados todos os funcionários que não foram alocados nos departamentos um, dois ou três.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- OPERADOR NOT IN
SELECT NR_MATRICULA, NM_FUNCIONARIO, CD_DEPTO
FROM   T_SIP_FUNCIONARIO
WHERE  CD_DEPTO NOT IN (1,2,3);
```

Código-fonte 24 - Exemplo de código-fonte SQL – Comando SELECT com operador NOT IN()
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

	NR_MATRICULA	NM_FUNCIONARIO	CD_DEPTO
1	12354	JOSEFINA DE ALMEIDA	4
2	12355	LUCIANA DE ALMEIDA	4
3	12356	THIAGO DE ALMEIDA	4
4	12357	LARISSSA DE CAMARGO	5
5	12358	ANTONIO DE CAMARGO	5
6	12359	JOSE DE CAMARGO	5

Figura 24 - Resultado da consulta à tabela FUNCIONARIO – Operador NOT IN()
Fonte: Elaborado pela autora (2023)

O comando SELECT retornará apenas as colunas declaradas e as linhas que atendem a condição especificada (filtro) da tabela “FUNCIONARIO”.

Observe que no filtro utilizamos o operador especial IN(). Neste exemplo, em função do filtro, serão recuperados todos os funcionários admitidos nos anos de 2012 ou 2014.

```
-- OPERADOR IN
SELECT  NR_MATRICULA ,
        DT_ADMISSAO ,
        VL_SALARIO_MENSAL
FROM    T_SIP_FUNCIONARIO
WHERE   TO_CHAR(DT_ADMISSAO,'YYYY') IN ('2012','2014');
```

Código-fonte 25 - Exemplo de código-fonte SQL – Comando SELECT com operador IN()
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

	NR_MATRICULA	DT_ADMISSAO	VL_SALARIO_MENSAL
1	12345	15/09/12	5684,66
2	12350	05/06/14	6273,99
3	12352	10/08/12	2563,44

Figura 25 - Resultado da consulta à tabela FUNCIONARIO – Operador IN()
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Importante:

TO_CHAR (<conteudo>,<formato>): converte o conteúdo informado para uma *string*. Podemos utilizar para converter uma data/horário em uma *string*, em um formato especificado. Neste exemplo TO_CHAR(DT_ADMISSAO,'YYYY'), convertemos uma data em *string* e capturamos apenas o ano com quatro dígitos, em função do formato especificado.

- **Operador SQL IS NULL e IS NOT NULL:**

O operador especial ou SQL IS NULL e IS NOT NULL é utilizado para recuperar linhas, cujo valor da coluna é nulo ou não nulo, respectivamente.

O comando SELECT retornará apenas as colunas declaradas e as linhas que atendem a condição especificada (filtro) da tabela "PROJETO".

Observe que no filtro utilizamos o operador especial IS NOT NULL. Neste exemplo, em função do filtro, serão recuperados todos os projetos finalizados, ou seja, todos os projetos que possuem a data término informada.

```
-- OPERADOR IS NOT NULL
SELECT CD_PROJETO, NM_PROJETO, DT_INICIO, DT_TERMINO
  FROM T_SIP_PROJETO
 WHERE DT_TERMINO IS NOT NULL;
```

Código-fonte 26 - Exemplo de código-fonte SQL – comando SELECT com operador IS NOT NULL
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta através do comando "SELECT":

	CD_PROJETO	NM_PROJETO	DT_INICIO	DT_TERMINO
1	10001	PROJETO 1	11/10/16	19/05/21
2	10002	PROJETO 2	25/10/13	15/04/15
3	10005	PROJETO 5	28/09/14	15/09/18

Figura 26 - Resultado da consulta à tabela FUNCIONARIO – Operador IS NOT NULL
Fonte: Elaborado pela autora (2023)

O comando SELECT retornará apenas as colunas declaradas e as linhas que atendem a condição especificada (filtro) da tabela "PROJETO".

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Observe que no filtro utilizamos o operador especial IS NULL. Neste exemplo, em função do filtro, serão recuperados todos os projetos em andamento, ou seja, todos os projetos que não possuem a data término informada.

```
-- OPERADOR IS NULL
SELECT CD_PROJETO, NM_PROJETO, DT_INICIO, DT_TERMINO
  FROM T_SIP_PROJETO
 WHERE DT_TERMINO IS NULL;
```

Código-fonte 27 - Exemplo de código-fonte SQL – comando SELECT com operador IS NULL
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

	CD_PROJETO	NM_PROJETO	DT_INICIO	DT_TERMINO
1	10003	PROJETO 3	28/04/12	(null)
2	10004	PROJETO 4	25/03/17	(null)
3	10006	PROJETO 6	05/04/20	(null)

Figura 27 - Resultado da consulta à tabela FUNCIONARIO – Operador IS NULL
Fonte: Elaborado pela autora (2023)

Atenção: os operadores IS NULL e IS NOT NULL são utilizados para avaliar se uma coluna tem valor ou não, portanto, devem ser aplicados em situações em que temos colunas opcionais.

- **Operador SQL LIKE:**

O operador especial ou SQL LIKE é utilizado para recuperar linhas, quando se deseja procurar um padrão em uma *string*. Os padrões são especificados usando uma combinação de caracteres normais e os dois caracteres curingas a seguir:

  - **Sublinhado (_):** corresponde a um caractere em uma posição específica.
  - **Porcentagem (%):** corresponde a qualquer número de caracteres a partir da posição especificada.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

EXPRESÃO	DESCRIÇÃO
LIKE 'A%'	Todas as palavras que iniciem a letra A.
LIKE '%A'	Todas as palavras que terminem com a letra A.
LIKE '%A%'	Todas as palavras que tenham letra A em qualquer posição.
LIKE 'A_'	String de dois caracteres que tenham a primeira letra A e o segundo caractere seja qualquer um.
LIKE '_A'	String de dois caracteres cujo primeiro seja qualquer um e a última letra seja A.
LIKE '_A_'	String de três caracteres cuja segunda letra seja A independentemente do primeiro ou do último caractere.
LIKE '%A_'	Todas as palavras que tenham a letra A na penúltima posição e a última seja qualquer outro caractere.
LIKE '_A%'	Todos que tenham a letra A na segunda posição e o primeiro caractere seja qualquer um.

Figura 28 - Operadores Especiais ou Operadores SQL
Fonte: Elaborado pela autora (2017)

O comando SELECT retornará apenas as colunas declaradas e as linhas que atendem a condição especificada (filtro) da tabela “DEPARTAMENTO”.

Observe que no filtro utilizamos o operador especial LIKE. Neste exemplo, em função do filtro, serão recuperados todos os departamentos cujos nomes sejam iniciados com as letras ‘FINA’.

```
-- OPERADOR LIKE
SELECT CD_DEPTO, NM_DEPTO
  FROM T_SIP_DEPARTAMENTO
 WHERE NM_DEPTO LIKE 'FINA%';
```

Código-fonte 28 - Exemplo de código-fonte SQL – comando SELECT com operador LIKE
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

	CD_DEPTO	NM_DEPTO
1	1	FINANCEIRO

Figura 29 - Resultado da consulta à tabela DEPARTAMENTO – Operador LIKE
Fonte: Elaborado pela autora (2023)

O comando SELECT retornará apenas as colunas declaradas e as linhas que atendem a condição especificada (filtro) da tabela “DEPARTAMENTO”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Observe que no filtro utilizamos o operador especial LIKE. Neste exemplo, em função do filtro, serão recuperados todos os departamentos que tenham como segundo caractere do nome a letra ‘A’.

```
-- OPERADOR LIKE
SELECT CD_DEPTO, NM_DEPTO
FROM   T_SIP_DEPARTAMENTO
WHERE  NM_DEPTO LIKE '_A%';
```

Código-fonte 29 - Exemplo de código-fonte SQL – comando SELECT com operador LIKE
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

	CD_DEPTO	NM_DEPTO
1	4	FATURAMENTO

Figura 30 - Resultado da consulta à tabela DEPARTAMENTO – Operador LIKE
Fonte: Elaborado pela autora (2023)

O comando SELECT retornará apenas as colunas declaradas e as linhas que atendem a condição especificada (filtro) da tabela “DEPARTAMENTO”.

Observe que no filtro utilizamos o operador especial LIKE. Neste exemplo, em função do filtro, serão recuperados todos os departamentos cujo nome tenha as letras ‘UR’ como quarto e quinto caracteres, respectivamente.

```
-- OPERADOR LIKE
SELECT CD_DEPTO, NM_DEPTO
FROM   T_SIP_DEPARTAMENTO
WHERE  NM_DEPTO LIKE '___UR%';
```

Código-fonte 30 - Exemplo de código-fonte SQL – comando SELECT com operador LIKE
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

	CD_DEPTO	NM_DEPTO
1	4	FATURAMENTO
2	5	RECURSOS HUMANOS

Figura 31 - Resultado da consulta à tabela DEPARTAMENTO – Operador LIKE
Fonte: Elaborado pela autora (2023)

O comando SELECT retornará apenas as colunas declaradas e as linhas que atendem a condição especificada (filtro) da tabela “DEPARTAMENTO”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Observe que no filtro utilizamos o operador especial LIKE. Neste exemplo, em função do filtro, serão recuperados todos os departamentos que tenham as letras ‘ME’ em qualquer parte do nome.

```
-- OPERADOR LIKE
SELECT CD_DEPTO, NM_DEPTO
FROM   T_SIP_DEPARTAMENTO
WHERE  NM_DEPTO LIKE '%ME%'
```

Código-fonte 31 - Exemplo de código-fonte SQL – comando SELECT com operador LIKE
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

	CD_DEPTO	NM_DEPTO
1	4	FATURAMENTO

Figura 32 - Resultado da consulta à tabela DEPARTAMENTO – Operador LIKE
Fonte: Elaborado pela autora (2023)

1.7.4 Exemplo de consulta com vários operadores

O comando SELECT retornará apenas as colunas declaradas e as linhas que atendem a condição especificada (filtro) da tabela “FUNCIONARIO”.

Observe que no filtro utilizamos vários operadores já apresentados. Neste exemplo, em função do filtro, serão recuperados todos os funcionários nascidos entre 01/01/1980 e 30/12/2010, admitidos a partir de 01/01/2013, com salário inferior a R$ 2300.00 e que não estejam alocados no departamento de código igual a três.

```
-- Exemplo utilizando vários operadores
      SELECT   NR_MATRICULA    ,
               CD_DEPTO        ,
               NM_FUNCIONARIO  ,
               DT_NASCIMENTO   ,
               DT_ADMISSAO     ,
               VL_SALARIO_MENSAL
        FROM   T_SIP_FUNCIONARIO
       WHERE   DT_NASCIMENTO
                   BETWEEN
                       TO_DATE('01/01/1980','DD/MM/YYYY')
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                    AND
                        TO_DATE('30/12/2010','DD/MM/YYYY')
                AND
                 DT_ADMISSAO > TO_DATE('01/01/2013','DD/MM/YYYY')
                AND VL_SALARIO_MENSAL < 2300
                AND NOT CD_DEPTO = 3 ;
```

Código-fonte 32 - Exemplo de código-fonte SQL – comando SELECT, vários operadores
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

	NR_MATRICULA	CD_DEPTO	NM_FUNCIONARIO	DT_NASCIMENTO	DT_ADMISSAO	VL_SALARIO_MENSAL
1	12347	1	JANDIRA DA SILVA	10/12/00	15/09/18	1875,96
2	12349	2	MARIA DAS DORES SOUZA	18/08/83	23/10/17	1647,44
3	12356	4	THIAGO DE ALMEIDA	10/03/98	24/10/18	1254,22
4	12357	5	LARISSSA DE CAMARGO	14/02/97	04/08/15	1245,55

Figura 33 - Resultado da consulta à tabela FUNCIONARIO – Vários operadores
Fonte: Elaborado pela autora (2023)

1.8 Operador de concatenação (||)

Esse operador permite a concatenação de colunas ou *string* de caracteres com outras colunas. Representamos o operador através de duas colunas verticais “||”. A coluna resultante de uma consulta, em que utilizamos este operador, é uma expressão de caracteres.

Exemplo de consulta utilizando operador de concatenação. Será apresentado o texto “O funcionário <nome> foi admitido em <data admissão>”, onde <nome> e <data admissão> serão obtidos a partir da leitura das colunas da tabela ‘FUNCIONARIO”, as demais informações representam uma *string* de caracteres que será concatenada aos campos, formando a frase desejada.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- EXIBIR O TEXTO: "O FUNCIONARIO <NOME> FOI
-- ADMITIDO EM <DATA ADMISSAO>"
SELECT NR_MATRICULA,
       'O FUNCIONARIO ' ||
       NM_FUNCIONARIO ||
       ', FOI ADMITIDO EM: ' ||
       DT_ADMISSAO "TEXTO"
  FROM T_SIP_FUNCIONARIO;
```

Código-fonte 33 - Exemplo de código-fonte SQL – comando SELECT com operador de concatenação
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

	NR_MATRICULA	TEXTO
1	12345	O FUNCIONARIO JOAO DA SILVA, FOI ADMITIDO EM: 15/09/12
2	12346	O FUNCIONARIO MANUEL DA SILVA, FOI ADMITIDO EM: 10/11/15
3	12347	O FUNCIONARIO JANDIRA DA SILVA, FOI ADMITIDO EM: 15/09/18
4	12348	O FUNCIONARIO KATIA REGINA SOUZA, FOI ADMITIDO EM: 03/10/15
5	12349	O FUNCIONARIO MARIA DAS DORES SOUZA, FOI ADMITIDO EM: 23/10/17
6	12350	O FUNCIONARIO ALFREDO DE SOUZA, FOI ADMITIDO EM: 05/06/14
7	12351	O FUNCIONARIO GISELE DE JESUS, FOI ADMITIDO EM: 20/03/17
8	12352	O FUNCIONARIO RAFAEL DE JESUS, FOI ADMITIDO EM: 10/08/12
9	12353	O FUNCIONARIO ROSANA DE JESUS, FOI ADMITIDO EM: 15/08/19
10	12354	O FUNCIONARIO JOSEFINA DE ALMEIDA, FOI ADMITIDO EM: 25/03/13
11	12355	O FUNCIONARIO LUCIANA DE ALMEIDA, FOI ADMITIDO EM: 28/09/11
12	12356	O FUNCIONARIO THIAGO DE ALMEIDA, FOI ADMITIDO EM: 24/10/18
13	12357	O FUNCIONARIO LARISSSA DE CAMARGO, FOI ADMITIDO EM: 04/08/15
14	12358	O FUNCIONARIO ANTONIO DE CAMARGO, FOI ADMITIDO EM: 12/08/16
15	12359	O FUNCIONARIO JOSE DE CAMARGO, FOI ADMITIDO EM: 20/04/17

Figura 34 - Resultado da consulta à tabela FUNCIONARIO – operador de concatenação
Fonte: Elaborado pela autora (2023)

1.9 Pseudocoluna ROWNUM

ROWNUM é uma expressão simples no SGBD Oracle. A pseudocoluna é utilizada quando precisamos limitar a quantidade de linhas retornadas através de um comando SELECT. No caso, a pseudocoluna ROWNUM numera os registros retornados por uma consulta.

Todas as tabelas possuem a pseudocoluna ROWNUM (retorna um número que indica a ordem em que o Oracle seleciona a linha a partir de uma tabela). Podemos utilizar, por exemplo, para recuperar um certo número de linhas por vez, para auxiliar a paginação em uma página WEB.

Exemplo de consulta utilizando a pseudocoluna ROWNUM.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- Exemplo pseudocoluna ROWNUM
SELECT   ROWNUM    ,
         CD_DEPTO ,
         NM_DEPTO
 FROM    T_SIP_DEPARTAMENTO ;
```

Código-fonte 34 - Exemplo de código-fonte SQL – comando SELECT, pseudocoluna ROWNUM
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

	ROWNUM	CD_DEPTO	NM_DEPTO
1	1	3	CONTAS A PAGAR
2	2	4	FATURAMENTO
3	3	1	FINANCEIRO
4	4	5	RECURSOS HUMANOS
5	5	2	TECNOLOGIA DA INFORMAÇÃO

Figura 35 - Resultado da consulta à tabela DEPARTAMENTO – pseudocoluna ROWNUM
Fonte: Elaborado pela autora (2023)

Exemplo de consulta utilizando a pseudocoluna ROWNUM, restringindo a quantidade de linhas recuperadas durante a consulta.

```
-- RESTRINGINDO A QUANTIDADE DE LINHAS RECUPERADAS
-- DURANTE A CONSULTA
SELECT     NR_MATRICULA ,
           CD_DEPTO ,
           DT_ADMISSAO ,
           VL_SALARIO_MENSAL ,
           (VL_SALARIO_MENSAL * 12) "SALÁRIO ANUAL"
  FROM    T_SIP_FUNCIONARIO
 WHERE   ROWNUM < 4;
```

Código-fonte 35 - Exemplo de código-fonte SQL – comando SELECT, pseudocoluna ROWNUM
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

	NR_MATRICULA	CD_DEPTO	DT_ADMISSAO	VL_SALARIO_MENSAL	SALÁRIO ANUAL
1	12345	1	15/09/12	5684,66	68215,92
2	12346	1	10/11/15	3542,11	42505,32
3	12347	1	15/09/18	1875,96	22511,52

Figura 36 - Resultado da consulta à tabela FUNCIONARIO – pseudocoluna ROWNUM
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.10 Análise TOP-N

Análise TOP-N é um tipo de consulta bastante útil em situações em que precisamos encontrar, por exemplo, os “N” maiores ou menores elementos dentro de determinado grupo: os funcionários mais bem pagos, os que vendem mais, as mercadorias menos vendidas e assim por diante.

Para conseguirmos isso no Oracle, devemos implementar uma consulta interna (subquery) que retornará todos os registros/linhas necessários, e na consulta externa limitaremos a exibição em um total desejado. Este processo é conhecido como consulta TOP-N no Oracle.

Exemplo de consulta utilizando análise TOP-N. A subquery (consulta interna) recupera todos os salários de funcionários em ordem decrescente (maior para o menor). A consulta externa apresenta os dados retornados da consulta interna, limitando nas três primeiras linhas. A limitação é feita através da cláusula WHERE, na qual validamos a quantidade de linhas recuperadas em um total menor ou igual a três.

```
-- PARA EXIBIR OS NOMES E OS SALÁRIOS DOS
-- TRÊS FUNCIONÁRIOS MAIS BEM REMUNERADOS
-- DA TABELA FUNCIONARIO.
      SELECT  ROWNUM as RANK ,
                     NM_FUNCIONARIO ,
                     VL_SALARIO_MENSAL
      FROM
          (    SELECT NM_FUNCIONARIO ,
                      VL_SALARIO_MENSAL
                 FROM T_SIP_FUNCIONARIO
             ORDER BY VL_SALARIO_MENSAL DESC
          )
      WHERE ROWNUM <= 3;
```

Código-fonte 36 - Exemplo de código-fonte SQL – comando SELECT, análise TOP-N
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta por meio do comando “SELECT”:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	RANK	NM_FUNCIONARIO	VL_SALARIO_MENSAL
1	1	JOSE DE CAMARGO	6541,22
2	2	ALFREDO DE SOUZA	6273,99
3	3	JOAO DA SILVA	5684,66

Figura 37 - Resultado da consulta à tabela FUNCIONARIO – análise TOP-N
Fonte: Elaborado pela autora (2023)

1.11 Configurando parâmetros NLS (National Language Support)

Os parâmetros NLS determinam o comportamento específico do local no cliente e no servidor. Vamos configurar alguns parâmetros de sessão para facilitar algumas visualizações das recuperações de dados.

Os parâmetros “ALTER SESSION” podem ser utilizados para substituir os padrões definidos para uma sessão de arquivo de inicialização ou definido pelo cliente com definições de ambiente.

Podemos visualizar a configuração de todos os atributos NLS, através do comando:

```
-- VISUALIZAR A CONFIGURAÇÃO ATUAL DOS ATRIBUTOS NLS
-- ATUAIS PARA A SUA SESSÃO
SELECT * FROM V$NLS_PARAMETERS;
```

Código-fonte 37 - Exemplo de código-fonte SQL – comando SELECT, atributos NLS
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

	PARAMETER	VALUE	CON_ID
1	NLS_LANGUAGE	BRAZILIAN PORTUGUESE	0
2	NLS_TERRITORY	BRAZIL	0
3	NLS_CURRENCY	R$	0
4	NLS_ISO_CURRENCY	BRAZIL	0
5	NLS_NUMERIC_CHARACTERS	,.	0
6	NLS_CALENDAR	GREGORIAN	0
7	NLS_DATE_FORMAT	DD/MM/RR	0
8	NLS_DATE_LANGUAGE	BRAZILIAN PORTUGUESE	0
9	NLS_CHARACTERSET	AL32UTF8	0
10	NLS_SORT	WEST EUROPEAN	0
11	NLS_TIME_FORMAT	HH24:MI:SSXFF	0
12	NLS_TIMESTAMP_FORMAT	DD/MM/RR HH24:MI:SSXFF	0
13	NLS_TIME_TZ_FORMAT	HH24:MI:SSXFF TZR	0
14	NLS_TIMESTAMP_TZ_FORMAT	DD/MM/RR HH24:MI:SSXFF TZR	0
15	NLS_DUAL_CURRENCY	Cr$	0
16	NLS_NCHAR_CHARACTERSET	AL16UTF16	0
17	NLS_COMP	BINARY	0
18	NLS_LENGTH_SEMANTICS	BYTE	0
19	NLS_NCHAR_CONV_EXCP	FALSE	0

Figura 38 - Resultado da consulta à tabela V$NLS_PARAMETERS – atributos NLS
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Exemplificaremos a alteração de sessão para o formato de data igual a “DD/MM/YYYY”.

Todas as consultas realizadas após essa alteração serão visualizadas com o novo formato:

```
-- Para alterar o formato de exibição de uma data
-- é utilizado a sintaxe:
ALTER SESSION SET NLS_DATE_FORMAT = 'DD/MM/YYYY';
```

Código-fonte 38 - Exemplo de código-fonte SQL – comando SELECT, atributos NLS_DATE_FORMAT
Fonte: Elaborado pela autora (2023)

Exemplo de consulta a tabela “PROJETO”, para visualizar a modificação do atributo “NLS_DATE_FORMAT”.

```
-- CONSULTA A TABELA PROJETO
SELECT * FROM T_SIP_PROJETO;
```

Código-fonte 39 - Exemplo de código-fonte SQL – comando SELECT
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

	CD_PROJETO	NM_PROJETO	DT_INICIO	DT_TERMINO
1	10001	PROJETO 1	11/10/2016	19/05/2021
2	10002	PROJETO 2	25/10/2013	15/04/2015
3	10003	PROJETO 3	28/04/2012	(null)
4	10004	PROJETO 4	25/03/2017	(null)
5	10005	PROJETO 5	28/09/2014	15/09/2018
6	10006	PROJETO 6	05/04/2020	(null)

Figura 39 - Resultado da consulta à tabela PROJETO após alteração do NLS_DATE_FORMAT
Fonte: Elaborado pela autora (2023)

Constantes que podem ser utilizadas na configuração do atributo “NLS_DATE_FORMAT”:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

YY = ano com 2 dígitos;
YYYY = ano com 4 dígitos;
RR = ano com 2 dígitos, com determinação do século conforme regras;
RRRR = ano com 4 dígitos, para entrada com 2 dígitos haverá determinação do século;
MM = o número do mês;
MONTH = o nome completo do mês;
MON = o nome abreviado do mês (jan, fev, mar e etc);
DDD = o dia do ano;
DD = o dia do mês;
D = o dia da semana;
DAY = o nome do dia;
HH = a hora do dia, no formato de 12 horas;
HH24= a hora do dia, no formato 24 horas;
MI = os minutos;
SS = os segundos;
SS.FF ou SSXFF = segundos com frações de segundo.

Figura 40 - Constantes para configuração do atributo NLS_DATE_FORMAT
Fonte: Elaborado pela autora (2023)

Exemplificaremos a alteração de sessão, para o idioma da data para a sessão corrente.

Todas as consultas realizadas após esta alteração serão visualizadas com o novo formato.

```
--Para alterar o idioma da data de exibição
-- é utilizado a sintaxe:
ALTER SESSION SET NLS_DATE_LANGUAGE = 'PORTUGUESE';
```

Código-fonte 40 - Exemplo de código-fonte SQL – comando SELECT, atributos NLS_DATE_LANGUAGE
Fonte: Elaborado pela autora (2023)

Exemplo de consulta a tabela “DUAL”, para visualizar a modificação do atributo “NLS_DATE_LANGUAGE”.

```
-- CONSULTA A DATA SO SERVIDOR
SELECT SYSDATE,
       TO_CHAR(SYSDATE, 'MONTH')
  FROM DUAL;
```

Código-fonte 41 - Exemplo de código-fonte SQL – comando SELECT
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Resultado da execução a consulta, através do comando “SELECT”:

	SYSDATE	TO_CHAR(SYSDATE,'MONTH')
1	25/01/2023	JANEIRO

Figura 41 - Resultado da consulta a data do sistema após alteração do NLS_DATE_LANGUAGE
Fonte: Elaborado pela autora (2023)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 HANDS ON – CONSULTA OU RECUPERAÇÃO DE DADOS

2.1 Estudo de Caso: SIP – SISTEMA DE CONTROLE DE IMPLANTACAO DE PROJETOS

2.1.1 Recuperando dados com comandos DRS ou DQL

No tópico “Exemplos de aplicação do comando SELECT” do capítulo, utilizamos o “SCRIPT DDL” e “SCRIPT DL” para criar a estrutura do estudo de caso SIP e popular as tabelas desse projeto.

Iremos consultar os dados das tabelas deste estudo de caso, para fixarmos os comandos trabalhados nas seções anteriores.

Como sugestão, podemos utilizar a ferramenta SQL Developer da Oracle.

O nome do SCRIPT (arquivo com comandos SQL) deve possuir um nome que ajude na sua identificação.

Por exemplo: “SCRIPT_DRS_PROJETO_SIP.SQL”, com a extensão SQL.

Consulta 1:

- Escreva e execute a instrução SQL, para realizar a consulta na tabela PROJETO.
- Liste todos os projetos finalizados, a partir de 01/08/2018.
- A lista deve ser apresentada de forma ordenada pela data término do projeto, em ordem decrescente.
- Mostrar: código, nome, data início e data término do projeto.
- Apresentar a coluna data término com o apelido “Finalizado em:”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- CONSULTA 1
  SELECT CD_PROJETO, NM_PROJETO,
         DT_INICIO, DT_TERMINO "Finalizado em:"
    FROM T_SIP_PROJETO
   WHERE DT_TERMINO IS NOT NULL AND
         DT_TERMINO > TO_DATE('01/08/2018','DD/MM/YYYY')
ORDER BY DT_TERMINO DESC;
```

Código-fonte 42 - Exemplo de código-fonte SQL – comando SELECT – Consulta 1
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

	CD_PROJETO	NM_PROJETO	DT_INICIO	Finalizado em:
1	10001	PROJETO 1	11/10/2016	19/05/2021
2	10005	PROJETO 5	28/09/2014	15/09/2018

Figura 42 - Resultado da Consulta 1
Fonte: Elaborado pela autora (2023)

Consulta 2:

- Escreva e execute a instrução SQL, para realizar a consulta na tabela IMPLANTACAO.
- Liste todas as implantações que possuem funcionários ativos, realizando atividades para um determinado projeto.
- Um funcionário ativo, é aquele que não terminou uma implantação.
- A lista deve ser apresentada de forma ordenada por código do projeto, número de matrícula do funcionário e data de entrada, todos crescentes.
- Mostrar: código da implantação, código do projeto, número de matrícula do funcionário, data de entrada e data de saída do funcionário.

```
-- CONSULTA 2
  SELECT *
    FROM T_SIP_IMPLANTACAO
   WHERE DT_SAIDA IS NULL
ORDER BY CD_PROJETO, NR_MATRICULA, DT_ENTRADA;
```

Código-fonte 43 - Exemplo de código-fonte SQL – comando SELECT – Consulta 2
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Resultado da execução a consulta, através do comando “SELECT”:

	CD_PROJETO	CD_IMPLANTACAO	NR_MATRICULA	DT_ENTRADA	DT_SAIDA
1	10001	2	12346	11/10/2016	(null)
2	10003	3	12345	28/04/2013	(null)
3	10003	1	12354	28/04/2013	(null)
4	10003	2	12355	28/04/2013	(null)
5	10004	1	12357	25/03/2017	(null)
6	10004	2	12358	25/03/2017	(null)

Figura 43 - Resultado da Consulta 2
Fonte: Elaborado pela autora (2023)

Consulta 3

- Escreva e execute a instrução SQL para realizar a consulta na tabela IMPLANTACAO.
- Liste todas as implantações finalizadas por funcionários.
- A lista deve ser apresentada de forma ordenada por código do projeto, número de matrícula do funcionário em ordem crescente e data da saída em ordem decrescente.
- Mostrar: código da implantação, código do projeto, número de matrícula do funcionário, data de entrada e data de saída do funcionário.
- Apresentar a coluna data entrada, com o apelido “Início em:”.
- Apresentar a coluna data saída, com o apelido “Conclusão:”.

```
-- CONSULTA 3
   SELECT CD_IMPLANTACAO, CD_PROJETO, NR_MATRICULA,
          DT_ENTRADA "Início em:",
          DT_SAIDA "Conclusão:"
    FROM T_SIP_IMPLANTACAO
   WHERE DT_SAIDA IS NOT NULL
ORDER BY CD_PROJETO, NR_MATRICULA, DT_SAIDA DESC;
```

Código-fonte 44 - Exemplo de código-fonte SQL – comando SELECT – Consulta 3
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Resultado da execução a consulta, através do comando “SELECT”:

	CD_IMPLANTACAO	CD_PROJETO	NR_MATRICULA	Início em:	Conclusão:
1	1	10001	12345	11/10/2016	15/09/2017
2	3	10001	12348	11/10/2016	25/06/2018
3	1	10002	12345	25/10/2013	15/04/2014
4	2	10002	12352	25/10/2013	25/07/2014
5	3	10002	12355	25/10/2013	15/04/2015
6	3	10004	12354	25/03/2017	15/08/2018
7	1	10005	12345	28/09/2014	15/09/2018
8	2	10005	12350	28/09/2014	15/09/2018
9	3	10005	12354	28/09/2014	15/09/2018

Figura 44 - Resultado da Consulta 3
Fonte: Elaborado pela autora (2023)

Consulta 4

- Escreva e execute a instrução SQL, para realizar a consulta na tabela IMPLANTACAO.
- Liste quais funcionários participam ou já participaram de implantações (em andamento ou finalizadas).
- Apresente o número de matrícula do funcionário, sem repetição.
- Mostrar: número de matrícula do funcionário.
- Apresentar a coluna número de matrícula do funcionário, com o apelido “Funcionário”.

```
-- CONSULTA 4
  SELECT DISTINCT NR_MATRICULA "Funcionário"
    FROM T_SIP_IMPLANTACAO;
```

Código-fonte 45 - Exemplo de código-fonte SQL – comando SELECT – Consulta 4
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	Funcionário
1	12357
2	12354
3	12352
4	12346
5	12350
6	12348
7	12355
8	12345
9	12358

Figura 45 - Resultado da Consulta 4
Fonte: Elaborado pela autora (2023)

Consulta 5:

- Escreva e execute a instrução SQL para realizar a consulta na tabela FUNCIONÁRIO.
- Liste os 5 funcionários com menor salário.
- Mostrar: rank, número de matrícula do funcionário, nome e salário mensal.
- Apresentar a pseudocoluna ROWUM, com o apelido “RANK”.

```
-- CONSULTA 5
SELECT ROWNUM AS RANK,
       NR_MATRICULA, NM_FUNCIONARIO, VL_SALARIO_MENSAL
FROM
        (SELECT NR_MATRICULA,
                NM_FUNCIONARIO, VL_SALARIO_MENSAL
           FROM T_SIP_FUNCIONARIO
         ORDER BY VL_SALARIO_MENSAL)
WHERE ROWNUM <=6;
```

Código-fonte 46 - Exemplo de código-fonte SQL – comando SELECT – Consulta 5
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	RANK	NR_MATRICULA	NM_FUNCIONARIO	VL_SALARIO_MENSAL
1	1	12357	LARISSSA DE CAMARGO	1245,55
2	2	12356	THIAGO DE ALMEIDA	1254,22
3	3	12351	GISELE DE JESUS	1360,66
4	4	12349	MARIA DAS DORES SOUZA	1647,44
5	5	12347	JANDIRA DA SILVA	1875,96
6	6	12355	LUCIANA DE ALMEIDA	2345,52

Figura 46 - Resultado da Consulta 5
Fonte: Elaborado pela autora (2023)

Consulta 6:

- Escreva e execute a instrução SQL, para realizar a consulta na tabela IMPLANTACAO.
- Liste todos as implantações ativas, ou seja, aquelas que o funcionário não finalizou as atividades.
- Nesta lista, deve ser apresentado o tempo em que cada implantação está em andamento, através do cálculo:

Tempo da implantação = SYSDATE – Data de entrada

- Apresentar uma mensagem com o seguinte texto: "TEMPO DA IMPLANTAÇÃO (<CÁLCULO DO TEMPO EM QUE ESTÃO EM ANDAMENTO>) DIAS."
- A lista deve ser apresentada de forma ordenada pelo tempo de implantação, em ordem decrescente.
- Apresentar a coluna mensagem, com o apelido “Tempo de Implantação”.
- Utilizar o operador para concatenação “||”, para concatenar o texto ao conteúdo dos campos.

Utilizar a função TO_CHAR(<conteúdo>, <formato>) para formatar o resultado do cálculo entre as duas datas, no momento da apresentação dos dados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- CONSULTA 6
  SELECT CD_PROJETO ,
         CD_IMPLANTACAO ,
         NR_MATRICULA,
         DT_ENTRADA,
         DT_SAIDA,
         ( 'TEMPO DA IMPLANTAÇÃO ( ' ||
           TO_CHAR(SYSDATE - DT_ENTRADA) ||
           ' ) DIAS.'
         ) "TEMPO DE IMPLANTAÇAO"
    FROM T_SIP_IMPLANTACAO
   WHERE DT_SAIDA IS NULL
ORDER BY "TEMPO DE IMPLANTAÇAO" DESC;
```

Código-fonte 47 - Exemplo de código-fonte SQL – comando SELECT – Consulta 6
Fonte: Elaborado pela autora (2023)

Resultado da execução a consulta, através do comando “SELECT”:

	CD_PROJETO	CD_IMPLANTACAO	NR_MATRICULA	DT_ENTRADA	DT_SAIDA	TEMPO DE IMPLANTAÇAO
1	10003	2	12355	28/04/2013	(null)	TEMPO DA IMPLANTAÇÃO (3559,9591550925925925925925925925925925) DIAS.
2	10003	3	12345	28/04/2013	(null)	TEMPO DA IMPLANTAÇÃO (3559,9591550925925925925925925925925925) DIAS.
3	10003	1	12354	28/04/2013	(null)	TEMPO DA IMPLANTAÇÃO (3559,9591550925925925925925925925925925) DIAS.
4	10001	2	12346	11/10/2016	(null)	TEMPO DA IMPLANTAÇÃO (2297,9591550925925925925925925925925925) DIAS.
5	10004	2	12358	25/03/2017	(null)	TEMPO DA IMPLANTAÇÃO (2132,9591550925925925925925925925925925) DIAS.
6	10004	1	12357	25/03/2017	(null)	TEMPO DA IMPLANTAÇÃO (2132,9591550925925925925925925925925925) DIAS.

Figura 47 - Resultado da Consulta 6
Fonte: Elaborado pela autora (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**: Fundamentos e Aplicações. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

HEUSER, C. A. **Projeto de Banco de Dados. Série Livros Didáticos**. v. 4. 6. ed. Rio Grande do Sul: Bookman, 2009.

MACHADO, Felipe Nery R. **Banco de Dados - Projeto e Implementação**. 2. ed. São Paulo: Érica, 2008.

PRICE, J. **Oracle Database 11g SQL**. São Paulo: Bookman, 2009.

PUGA, S.; FRANÇA, E.; GOYA, M. **Banco de Dados: Implementação em SQL, PL/SQL e Oracle 11g**. São Paulo: Pearson Education do Brasil, 2013.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 5. ed. Rio de Janeiro: Campus, 2006.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

DRS	Data Retrieve Language (Linguagem de recuperação de dados).
DQL	Data Query Language (Linguagem para consulta de dados).
SELECT	Comando da linguagem SQL responsável por permitir a consulta ou recuperação de registros ou linhas nas tabelas.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com